Assignment 1 Report

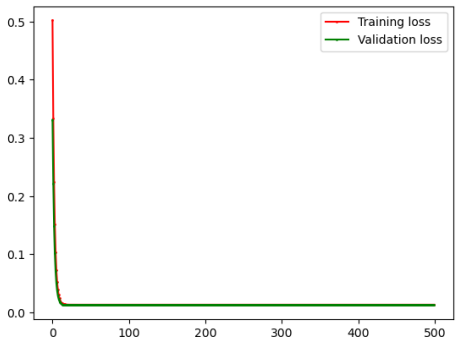
1. Linear Regression (20 points):

a. **Change the learning rate to the following values, and plot the loss curve for each learning rate:**

i. linear_data_A

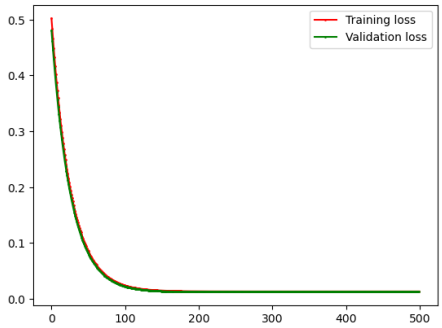
1. learning rate 0.1

```
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
✓ 0.0s

50. Training loss: 0.013423623061455565, Val loss:0.012580010701461058
100. Training loss: 0.013423622480987135, Val loss:0.012580089713127157
150. Training loss: 0.013423622480987133, Val loss:0.012580089715050587
200. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
250. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
300. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
350. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
400. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
450. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
500. Training loss: 0.013423622480987133, Val loss:0.012580089715050636
```



2. learning rate 0.01

```
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
✓ 0.0s

50. Training loss: 0.0882249976145924, Val loss:0.08344253861678508
100. Training loss: 0.024461533163063466, Val loss:0.022730598385935274
150. Training loss: 0.015054341888577379, Val loss:0.01396118427162712
200. Training loss: 0.013664791563666536, Val loss:0.012739408114859366
250. Training loss: 0.013459321509937406, Val loss:0.012586753610002818
300. Training loss: 0.013428910982968287, Val loss:0.012574716912916001
350. Training loss: 0.013424406462249211, Val loss:0.01257690199190865
400. Training loss: 0.013423738769333433, Val loss:0.012578717009069202
450. Training loss: 0.013423639738964216, Val loss:0.012579546976647175
500. Training loss: 0.013423625043327498, Val loss:0.01257988125786218
```

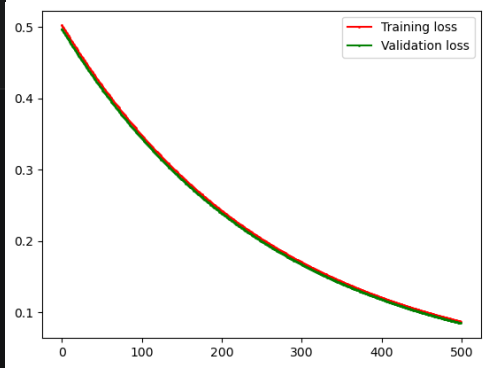


3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.418984925274338, Val loss:0.41420880497204005
100. Training loss: 0.34884281364937, Val loss:0.3445251797643867
150. Training loss: 0.2908358402579774, Val loss:0.28691334687110626
200. Training loss: 0.24286379534660618, Val loss:0.23928338706190333
250. Training loss: 0.20319008395293756, Val loss:0.19990713434668128
300. Training loss: 0.17037874765339447, Val loss:0.1673555525025328
350. Training loss: 0.14324239869173927, Val loss:0.14044695514816508
400. Training loss: 0.12079917484937544, Val loss:0.11820419110090796
450. Training loss: 0.10223715148116816, Val loss:0.09981924222927957
500. Training loss: 0.08688491828078362, Val loss:0.08462395006505138
```



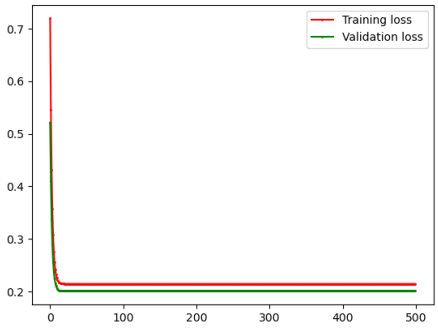
ii. linear_data_B

1. learning rate 0.1

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.2147779602976193, Val loss:0.20128111574984847
100. Training loss: 0.21477795969579408, Val loss:0.20128143543301202
150. Training loss: 0.2147779596957941, Val loss:0.20128143544080995
200. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
250. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
300. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
350. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
400. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
450. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
500. Training loss: 0.21477795969579408, Val loss:0.20128143544081017
```

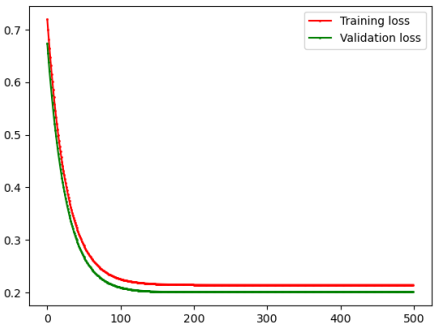


2. learning rate 0.01

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

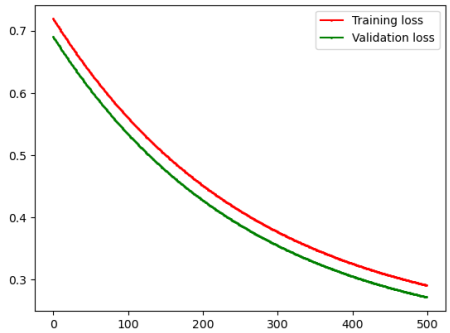
```
50. Training loss: 0.2920213425997664, Val loss:0.27026114086461656
100. Training loss: 0.2261837940753283, Val loss:0.2101879308116208
150. Training loss: 0.2164640141651331, Val loss:0.2021155812609517
200. Training loss: 0.21502743867502366, Val loss:0.20122361991141446
250. Training loss: 0.21481490508468085, Val loss:0.20120482979710497
300. Training loss: 0.21478343493349927, Val loss:0.20124452118342356
350. Training loss: 0.21477877162933795, Val loss:0.20126634873684507
400. Training loss: 0.21477808016505961, Val loss:0.20127557772119783
450. Training loss: 0.21477797757871428, Val loss:0.20127920153421996
500. Training loss: 0.2147779623514954, Val loss:0.20128058874123028
```



3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```
✓ 0.0s

```
50. Training loss: 0.6333011463329974, Val loss:0.605763731465133
100. Training loss: 0.560944176029853, Val loss:0.5347780170370381
150. Training loss: 0.5011005997675118, Val loss:0.4761621599409662
200. Training loss: 0.4516056474010013, Val loss:0.4277683253433775
250. Training loss: 0.410669175541332, Val loss:0.3878209660417229
300. Training loss: 0.37681081233745717, Val loss:0.35485234230464696
350. Training loss: 0.3488063343172151, Val loss:0.32764920718227697
400. Training loss: 0.32564332924740713, Val loss:0.3052087237083345
450. Training loss: 0.3064845362852951, Val loss:0.28670201534712025
500. Training loss: 0.2906375335120469, Val loss:0.27144402793270805
```

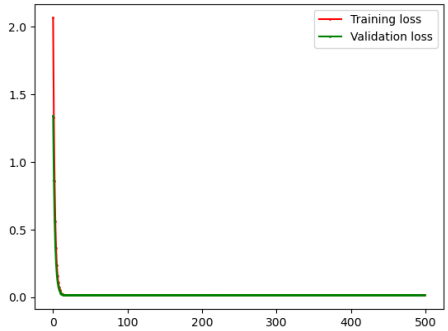


iii. linear_data_C

1. learning rate 0.1

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
```
✓ 0.0s

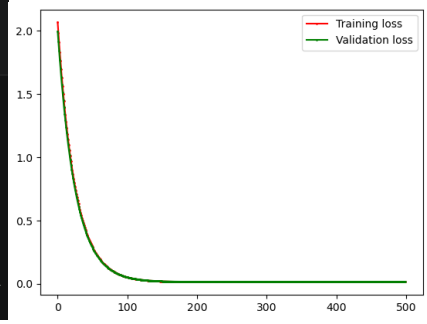
```
50. Training loss: 0.0131818971313023, Val loss:0.013771656751385359
100. Training loss: 0.013181896123422132, Val loss:0.013771595669720494
150. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
200. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
250. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
300. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
350. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
400. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
450. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
500. Training loss: 0.013181896123422132, Val loss:0.013771595669175307
```



2. learning rate 0.01

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```
✓ 0.0s

```
50. Training loss: 0.30167765721022727, Val loss:0.2940364008397446
100. Training loss: 0.05230128191456955, Val loss:0.05228622570263863
150. Training loss: 0.018505850726527058, Val loss:0.01917541216232359
200. Training loss: 0.01390898372103327, Val loss:0.014564896297918118
250. Training loss: 0.01328152198797338, Val loss:0.01389971244927915
300. Training loss: 0.01319558969139006, Val loss:0.013796096535113092
350. Training loss: 0.013183783898695744, Val loss:0.013777427806772765
400. Training loss: 0.013182157103470477, Val loss:0.013773277022245267
450. Training loss: 0.013181932299897786, Val loss:0.013772140690770074
500. Training loss: 0.013181901150900588, Val loss:0.013771782558303797
```

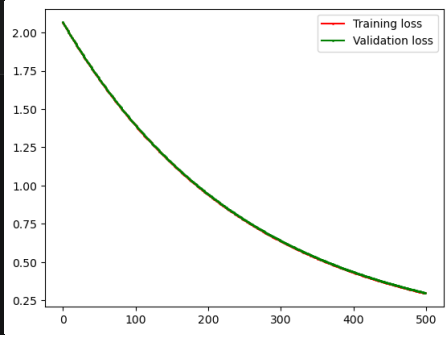


3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 1.7022307356186883, Val loss:1.7043995619027572
100. Training loss: 1.398099093422373, Val loss:1.4007211470145513
150. Training loss: 1.1487734111234604, Val loss:1.1516658664027397
200. Training loss: 0.9443693636081104, Val loss:0.9473976449419805
250. Training loss: 0.7767867515803093, Val loss:0.7798533335010578
300. Training loss: 0.6393871949572305, Val loss:0.6424228534106732
350. Training loss: 0.526730101508695, Val loss:0.529687097137462
400. Training loss: 0.4343563650291918 4, Val loss:0.4372031470837644
450. Training loss: 0.3586111574044349 4, Val loss:0.3613282625111079 4
500. Training loss: 0.2964987427402306, Val loss:0.2990756308422210 5
```



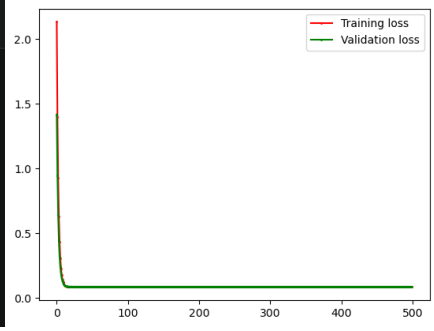
iv. linear_data_D

1. learning rate 0.1

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.08238685177710688, Val loss:0.08607262493707105
100. Training loss: 0.08238685077138833, Val loss:0.08607247293371127
150. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
200. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
250. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
300. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
350. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
400. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
450. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
500. Training loss: 0.08238685077138833, Val loss:0.08607247293234568
```

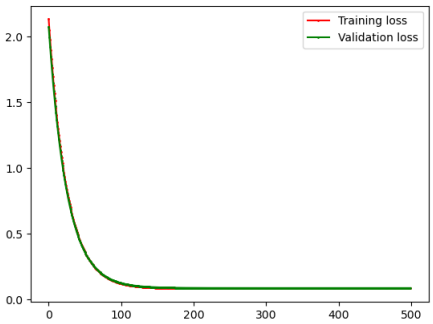


2. learning rate 0.01

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.3703643487722049, Val loss:0.3686849563983321
100. Training loss: 0.12143459698593069, Val loss:0.1255385955779745
150. Training loss: 0.08770086474185745, Val loss:0.09183236776167002
200. Training loss: 0.08311255449810843, Val loss:0.08699540377230673
250. Training loss: 0.08248628344524059, Val loss:0.08624729244656879
300. Training loss: 0.08240051730594523, Val loss:0.08611372904350847
350. Training loss: 0.08238873475630884, Val loss:0.08608430475529506
400. Training loss: 0.0823871112191, Val loss:0.08607630029028919
450. Training loss: 0.08238688687299428, Val loss:0.08607378489771042
500. Training loss: 0.0823868557883243, Val loss:0.08607293368416795
```

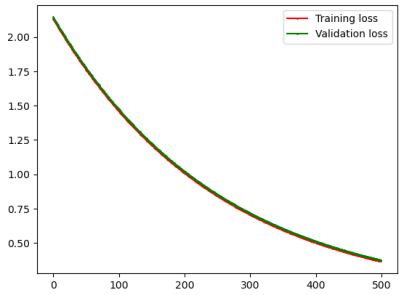


3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,True,loss_fn,grad_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 1.7684511643778449, Val loss:1.7808955296769526
100. Training loss: 1.4648524964143852, Val loss:1.4770554865390988
150. Training loss: 1.2159645145907358, Val loss:1.2278106296911495
200. Training loss: 1.0119199428934709, Val loss:1.0233353741451867
250. Training loss: 0.8446325767970521, Val loss:0.8555746535033422
300. Training loss: 0.707475524484535, Val loss:0.7179245466593899
350. Training loss: 0.5950176249327075, Val loss:0.6049705693586136
400. Training loss: 0.50280751484024, Val loss:0.5122732093945561
450. Training loss: 0.4271967232656893, Val loss:0.43619216922708653
500. Training loss: 0.3651947340371162, Val loss:0.37374232354720466
```



b. **Following 1(a), show the evaluations by using the "evaluate_linear_regression" function:**

i. linear_data_A

1. learning rate 0.1

```
=== Linear Regression Evaluation ===
MSE       : 0.0134
MAE       : 0.1010
RMSE      : 0.1156
R-squared : 0.5690

{'MSE': 0.013356333749635324,
 'MAE': 0.10095600779556413,
 'RMSE': 0.11556960564800471,
 'R-squared': 0.5689641130867413}
```

2. learning rate 0.01

```
=== Linear Regression Evaluation ===
MSE       : 0.0134
MAE       : 0.1010
RMSE      : 0.1156
R-squared : 0.5690

{'MSE': 0.013355229539356039,
 'MAE': 0.10095149880498926,
 'RMSE': 0.11556482829717717,
 'R-squared': 0.5689997481843656}
```

3. learning rate 0.001

```
=== Linear Regression Evaluation ===
MSE       : 0.0832
MAE       : 0.2369
RMSE      : 0.2884
R-squared : -1.6836

{'MSE': 0.0831564528379084,
 'MAE': 0.2369175723474691,
 'RMSE': 0.2883686058465942,
 'R-squared': -1.683626815070196}
```

ii. linear_data_B

1. learning rate 0.1

```
=== Linear Regression Evaluation ===
MSE       : 0.2137
MAE       : 0.4038
RMSE      : 0.4623
R-squared : 0.0448

{'MSE': 0.21370133999416518,
 'MAE': 0.4038240311822565,
 'RMSE': 0.46227842259201885,
 'R-squared': 0.04477536473581445}
```

2. learning rate 0.01

```
=== Linear Regression Evaluation ===
MSE       : 0.2137
MAE       : 0.4038
RMSE      : 0.4623
R-squared : 0.0448

{'MSE': 0.21369682365896048,
 'MAE': 0.40381943393609915,
 'RMSE': 0.4622735377013922,
 'R-squared': 0.04479555232401}
```

3. learning rate 0.001

```
=== Linear Regression Evaluation ===
MSE       : 0.2687
MAE       : 0.4404
RMSE      : 0.5184
R-squared : -0.2011

{'MSE': 0.26870180820833145,
 'MAE': 0.4403913458737695,
 'RMSE': 0.5183645514580751,
 'R-squared': -0.20107148952663834}
```

iii. linear_data_C

1. learning rate 0.1

```
=== Linear Regression Evaluation ===
MSE       : 0.0132
MAE       : 0.0987
RMSE      : 0.1148
R-squared : 0.9954

{'MSE': 0.013182793631671172,
 'MAE': 0.09868229952957776,
 'RMSE': 0.11481634740606919,
 'R-squared': 0.9953516857442621}
```

2. learning rate 0.01

```
=== Linear Regression Evaluation ===
MSE       : 0.0132
MAE       : 0.0987
RMSE      : 0.1148
R-squared : 0.9954

{'MSE': 0.01318285194198864,
 'MAE': 0.09868109058967203,
 'RMSE': 0.11481660133442655,
 'R-squared': 0.9953516651837735}
```

3. learning rate 0.001

```
=== Linear Regression Evaluation ===
MSE       : 0.2852
MAE       : 0.4273
RMSE      : 0.5341
R-squared : 0.8994

{'MSE': 0.28522489191953343,
 'MAE': 0.4273488119851382,
 'RMSE': 0.5340645016470702,
 'R-squared': 0.8994283785179172}
```

iv. linear_data_D

1. learning rate 0.1

```
=== Linear Regression Evaluation ===
MSE       : 0.0824
MAE       : 0.2467
RMSE      : 0.2870
R-squared : 0.9716

{'MSE': 0.08239246019794483,
 'MAE': 0.24670574882394442,
 'RMSE': 0.28704086851517296,
 'R-squared': 0.9716056317069087}
```

2. learning rate 0.01

```
=== Linear Regression Evaluation ===
MSE       : 0.0824
MAE       : 0.2467
RMSE      : 0.2870
R-squared : 0.9716

{'MSE': 0.08239259798771861,
 'MAE': 0.24670454246976223,
 'RMSE': 0.28704110853276504,
 'R-squared': 0.9716055842213308}
```

3. learning rate 0.001

```
=== Linear Regression Evaluation ===
MSE       : 0.3540
MAE       : 0.4778
RMSE      : 0.5950
R-squared : 0.8780

{'MSE': 0.35401418955181013,
 'MAE': 0.4777943162702676,
 'RMSE': 0.5949909155204054,
 'R-squared': 0.8779984326846808}
```

c. Describe your observations in 1(a) and 1(b).

Optimal Convergence : Across all four datasets, learning rates of 0.1 and 0.01 yield almost identical evaluation metrics (e.g., the same MSE and R-squared). The loss curves indicate that both rates are large enough to successfully converge to the minimum within the 500-iteration limit. Furthermore, no divergence is observed when lr = 0.1, meaning it is a safe and highly efficient step size for the scale of these datasets.

Slow Convergence / Underfitting : A learning rate of 0.001 consistently performs the worst across all datasets, resulting in higher error metrics (MSE, MAE, RMSE) and significantly lower R-squared values (even negative in datasets A and B). The loss curves confirm that the step size is too small, preventing the model from fully converging within 500 iterations and leading to an underfitted model.

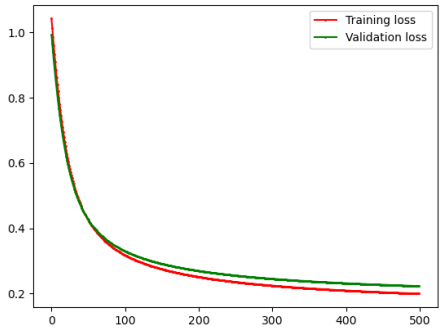
2. Logistic Regression (20 points):

a. Change the learning rate to the following values, and plot the loss curve for each learning rate:

i. logistic_data_A

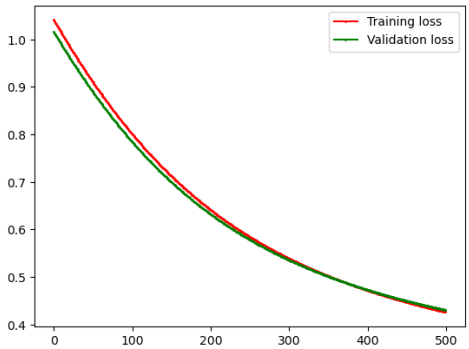
1. learning rate 0.1

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
✓ 0.0s

50. Training loss: 0.42788681156341524, Val loss:0.42893372331948243
100. Training loss: 0.3186543688258759, Val loss:0.330744022589277
150. Training loss: 0.2756136848389012, Val loss:0.2918366320362445
200. Training loss: 0.2515647449518196, Val loss:0.27001137334938924
250. Training loss: 0.23586370599134487, Val loss:0.2557354638551537
300. Training loss: 0.22467783764966348, Val loss:0.2455615161221097
350. Training loss: 0.21624915555159702, Val loss:0.23790131502512443
400. Training loss: 0.20964409595419928, Val loss:0.23190816484103666
450. Training loss: 0.2043156331315366, Val loss:0.22708432494974984
500. Training loss: 0.19991967899618934, Val loss:0.2231158675900225
```



2. learning rate 0.01

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
✓ 0.0s

50. Training loss: 0.9117697645477548, Val loss:0.8909436605779335
100. Training loss: 0.8023254524923707, Val loss:0.7857578532165119
150. Training loss: 0.7134127241759743, Val loss:0.700717653642279
200. Training loss: 0.6421019406042985, Val loss:0.6328939736163958
250. Training loss: 0.5851379734874116, Val loss:0.5790315939113776
300. Training loss: 0.5394700517290799, Val loss:0.536090659945112
350. Training loss: 0.5025287985878911, Val loss:0.5015278107602668
400. Training loss: 0.472286903656601, Val loss:0.4733528541640398
450. Training loss: 0.4472015131236836, Val loss:0.45006424591042526
500. Training loss: 0.42611745469430734, Val loss:0.43054684173876917
```

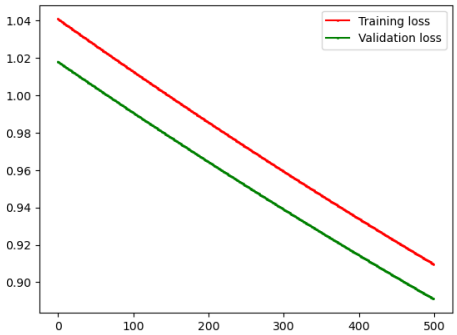


3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 1.0269460601586051, Val loss:1.0044831492028101
100. Training loss: 1.0129964397614701, Val loss:0.9909719737552948
150. Training loss: 0.9992783487558338, Val loss:0.9776880822134313
200. Training loss: 0.9857916009817304, Val loss:0.9646313599916956
250. Training loss: 0.9725358539101563, Val loss:0.9518015338881938
300. Training loss: 0.9595106080951762, Val loss:0.9391981713655225
350. Training loss: 0.9467152071471594, Val loss:0.9268206803770204
400. Training loss: 0.9341488382362758, Val loss:0.9146683097467531
450. Training loss: 0.921810533131396, Val loss:0.902740150107322
500. Training loss: 0.909699169775372, Val loss:0.8910351353952264
```



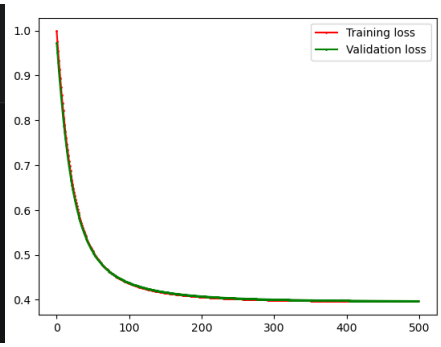
ii. logistic_data_B

1. learning rate 0.1

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.5103175622043802, Val loss:0.5083432819772871
100. Training loss: 0.4370955155097443, Val loss:0.4380558711885709
150. Training loss: 0.41557622016883045, Val loss:0.41686490530445874
200. Training loss: 0.4064609166991814, Val loss:0.4077246042337751
250. Training loss: 0.40189820214342353, Val loss:0.4030758565758929
300. Training loss: 0.39939491697142937, Val loss:0.4004811213417746
350. Training loss: 0.39794090751624056, Val loss:0.39894289148678264
400. Training loss: 0.3970632739456962, Val loss:0.3979910069463985
450. Training loss: 0.39651887299277666, Val loss:0.3973823809390223
500. Training loss: 0.39617431232724015, Val loss:0.39698286902644125
```

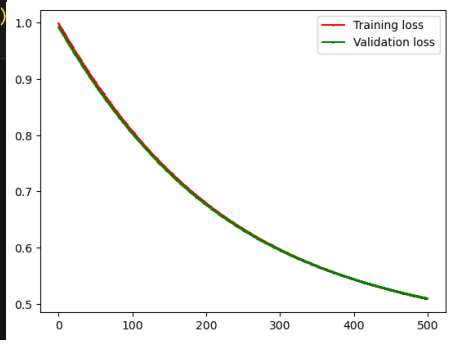


2. learning rate 0.01

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.8957410640363592, Val loss:0.8906374105058227
100. Training loss: 0.8083096467352318, Val loss:0.804210107205688
150. Training loss: 0.7369603264945955, Val loss:0.7337661725077593
200. Training loss: 0.6795500174801885, Val loss:0.6771732291658011
250. Training loss: 0.6336948605195072, Val loss:0.6320453148336889
300. Training loss: 0.5971080713785597, Val loss:0.5960905942359478
350. Training loss: 0.5678011822962918, Val loss:0.567319509722204
400. Training loss: 0.5441528833412146, Val loss:0.5441156176252377
450. Training loss: 0.5248927073071165, Val loss:0.5252178109592448
500. Training loss: 0.5090467133965937, Val loss:0.5096635682600803
```

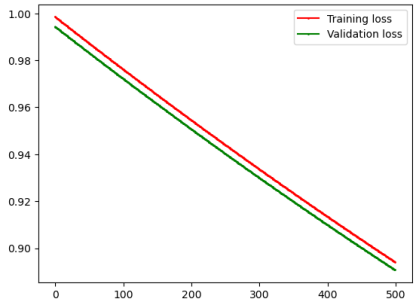


3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

```
50. Training loss: 0.9874379008463233, Val loss:0.9833381665312408
100. Training loss: 0.9763421106130916, Val loss:0.9723285004243568
150. Training loss: 0.9654284713499056, Val loss:0.9614993121069905
200. Training loss: 0.9546966109038486, Val loss:0.9508503118676399
250. Training loss: 0.9441460548822513, Val loss:0.9403811053688884
300. Training loss: 0.9337762268335361, Val loss:0.9300911936941738
350. Training loss: 0.9235864487116805, Val loss:0.9199799736931847
400. Training loss: 0.9135759416259137, Val loss:0.9100467386272083
450. Training loss: 0.9037438268754272, Val loss:0.900290679113813
500. Training loss: 0.8940891272670412, Val loss:0.890710884368335
```



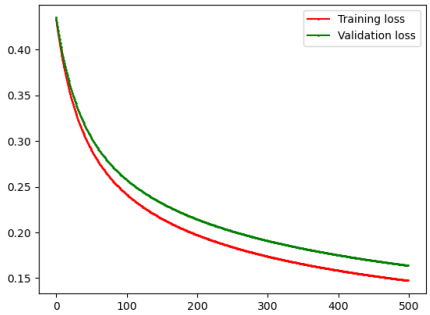
iii. logistic_data_C

1. learning rate 0.1

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
```

✓ 0.1s

```
50. Training loss: 0.2914041500436064, Val loss:0.30461404403017917
100. Training loss: 0.24213528492723757, Val loss:0.25808888358348686
150. Training loss: 0.21536654598435145, Val loss:0.2322484565856379
200. Training loss: 0.1974732668561366, Val loss:0.21464876637946464
250. Training loss: 0.18425966445090683, Val loss:0.20146358893625357
300. Training loss: 0.17392662515639434, Val loss:0.19104062396513338
350. Training loss: 0.1655356333407847, Val loss:0.18250642396014666
400. Training loss: 0.15853434715164158, Val loss:0.17533939085232825
450. Training loss: 0.1525707400848272, Val loss:0.1692025922688029
500. Training loss: 0.14740737629117945, Val loss:0.1638661602105938
```

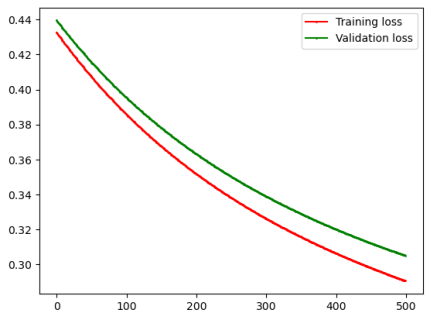


2. learning rate 0.01

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.1s

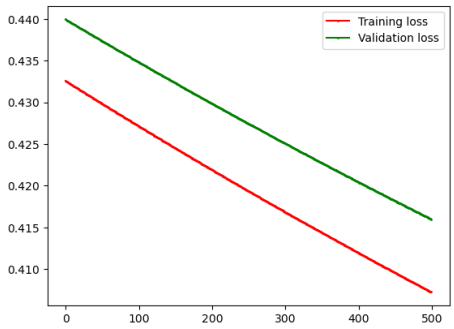
```
50. Training loss: 0.40763335808873047, Val loss:0.41596249839320293
100. Training loss: 0.3860913696555631, Val loss:0.3956206679931872
150. Training loss: 0.3677333549357604, Val loss:0.3782638648505192
200. Training loss: 0.35195423947335824, Val loss:0.36332831682999583
250. Training loss: 0.33827534357025274, Val loss:0.3503673663467325
300. Training loss: 0.32631769333124555, Val loss:0.3390266190729544
350. Training loss: 0.31578039047898315, Val loss:0.32902400827673756
400. Training loss: 0.3064235064986363, Val loss:0.3201340925514732
450. Training loss: 0.29805473954887807, Val loss:0.3121758074752029
500. Training loss: 0.2905190785492918, Val loss:0.3050029544730205
```



3. learning rate 0.001

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
✓ 0.1s
```

```
50. Training loss: 0.4298821030676496, Val loss:0.43740830867066577
100. Training loss: 0.4271883654785384, Val loss:0.43486105109553536
150. Training loss: 0.42454112656343235, Val loss:0.4323574765831896
200. Training loss: 0.42193938784995444, Val loss:0.42989664927305665
250. Training loss: 0.4193821724637837, Val loss:0.4274776538321253
300. Training loss: 0.41686852479007863, Val loss:0.4250995951031024
350. Training loss: 0.41439751012942966, Val loss:0.42276159774950733
400. Training loss: 0.4119682143493205, Val loss:0.4204628058985884
450. Training loss: 0.40957974353201126, Val loss:0.4182023827828775
500. Training loss: 0.40723122361969855, Val loss:0.4159795103811308
```

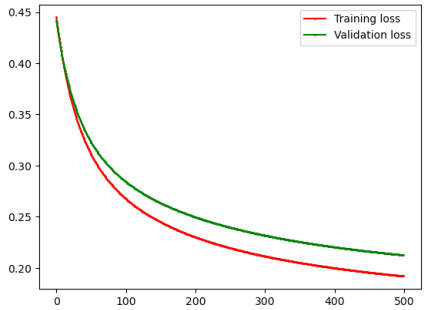


iv. logistic_data_D

1. learning rate 0.1

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.1,n_iteration =500,val_ratio=0.2)
✓ 0.1s
```

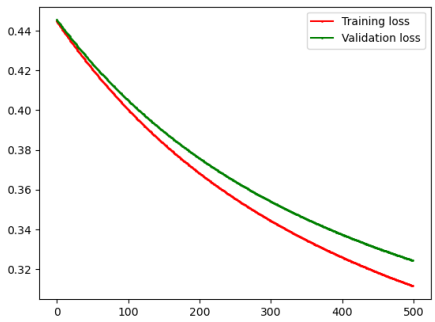
```
50. Training loss: 0.31214565926023174, Val loss:0.3238995705537948
100. Training loss: 0.26802535795005145, Val loss:0.2845901168491454
150. Training loss: 0.24507991801016674, Val loss:0.2636714373900576
200. Training loss: 0.23025970529698733, Val loss:0.24981367489682632
250. Training loss: 0.21963615020155505, Val loss:0.23967468756947907
300. Training loss: 0.2115547767275018, Val loss:0.2318441744405014
350. Training loss: 0.20516346620008113, Val loss:0.2255827485342569
400. Training loss: 0.1999656677345639, Val loss:0.22044961112666256
450. Training loss: 0.19564762834057503, Val loss:0.21616005232291835
500. Training loss: 0.19199945372601435, Val loss:0.21251993712270317
```



2. learning rate 0.01

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
✓ 0.1s
```

```
50. Training loss: 0.4210863364107005, Val loss:0.42389318391426317
100. Training loss: 0.4006650658061349, Val loss:0.40526939422934116
150. Training loss: 0.3833104146387553, Val loss:0.3894644471413472
200. Training loss: 0.36844391326627834, Val loss:0.37594512076571973
250. Training loss: 0.3556059025863166, Val loss:0.3642876353134937
300. Training loss: 0.3444310989408412, Val loss:0.354154961784246
350. Training loss: 0.33462857869617535, Val loss:0.3452784421002676
400. Training loss: 0.32596580182229623, Val loss:0.33744318725215144
450. Training loss: 0.3182560482209092, Val loss:0.33047660104180177
500. Training loss: 0.3113486162459987, Val loss:0.3242394100832966
```

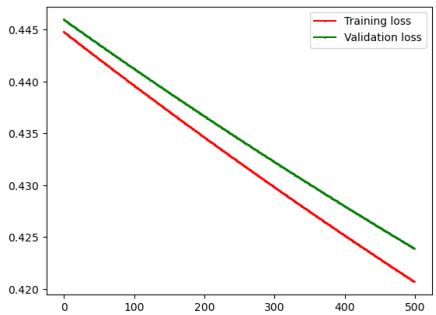


3. learning rate 0.001

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.001,n_iteration =500,val_ratio=0.2)
```

✓ 0.1s

```
50. Training loss: 0.44221977024873604, Val loss:0.4436168074363421
100. Training loss: 0.4396594302179652, Val loss:0.4412701641807759
150. Training loss: 0.4371436725613986, Val loss:0.4389647208426862
200. Training loss: 0.4346715536135045, Val loss:0.4366996003801621
250. Training loss: 0.4322421495919633, Val loss:0.43447394458012223
300. Training loss: 0.4298545563023576, Val loss:0.43228691374881156
350. Training loss: 0.42750788883734553, Val loss:0.43013768639915717
400. Training loss: 0.42520128127118983, Val loss:0.42802545893576116
450. Training loss: 0.4229338863504631, Val loss:0.4259494453382512
500. Training loss: 0.42070487518169253, Val loss:0.42390887684365186
```

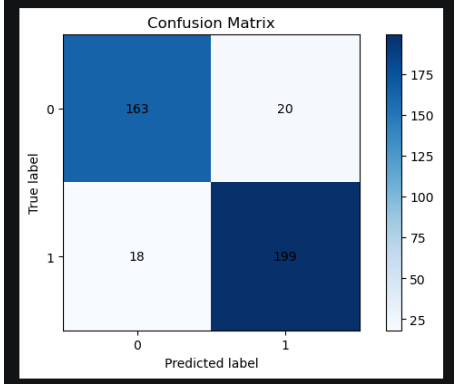


b. **Following 2(a), show the evaluations by using the "evaluate_binary_classifier" function.**

i. logistic_data_A

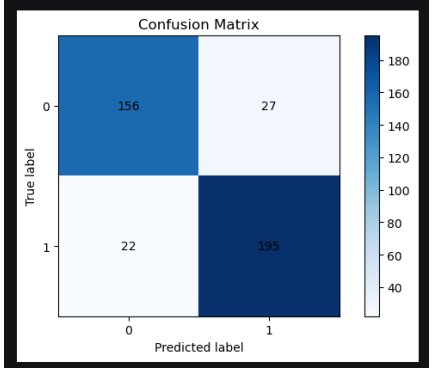
1. learning rate 0.1

```
Logistic Regression Evaluation
Accuracy  : 0.9050
Precision : 0.9087
Recall    : 0.9171
F1-score  : 0.9128
```



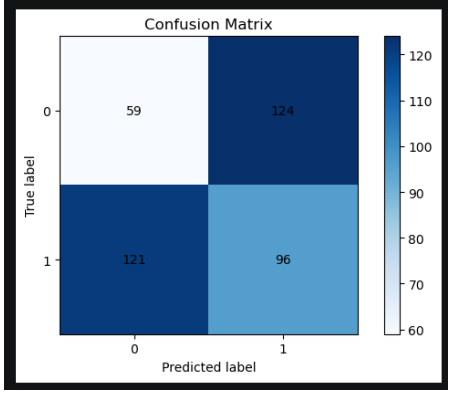
2. learning rate 0.01

```
Logistic Regression Evaluation
Accuracy  : 0.8775
Precision : 0.8784
Recall    : 0.8986
F1-score  : 0.8884
```



3. learning rate 0.001

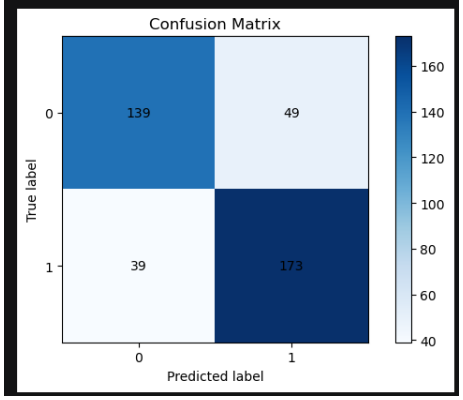
```
Logistic Regression Evaluation
Accuracy  : 0.3875
Precision : 0.4364
Recall    : 0.4424
F1-score  : 0.4394
```



ii. logistic_data_B

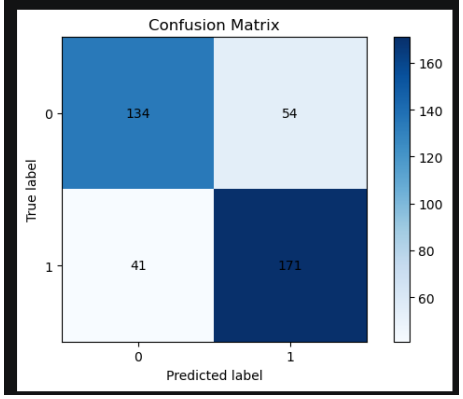
1. learning rate 0.1

```
Logistic Regression Evaluation
Accuracy  : 0.7800
Precision : 0.7793
Recall    : 0.8160
F1-score  : 0.7972
```

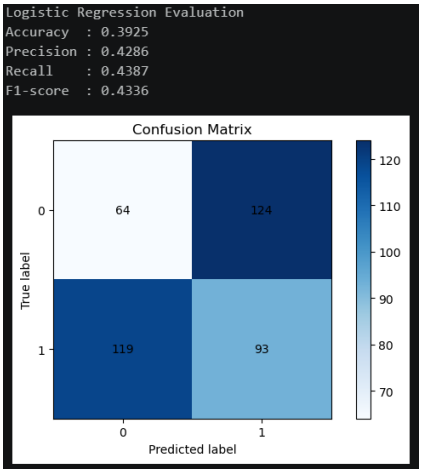


2. learning rate 0.01

```
Logistic Regression Evaluation
Accuracy  : 0.7625
Precision : 0.7600
Recall    : 0.8066
F1-score  : 0.7826
```

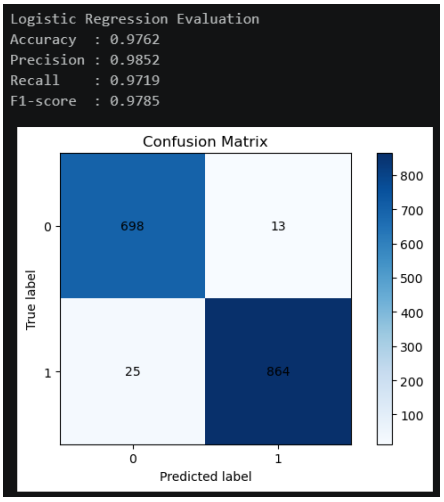


3. learning rate 0.001

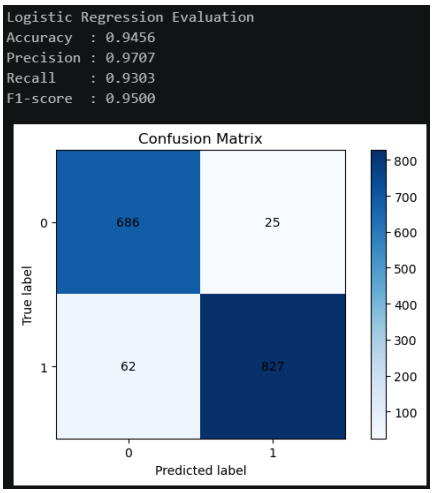


iii. logistic_data_C

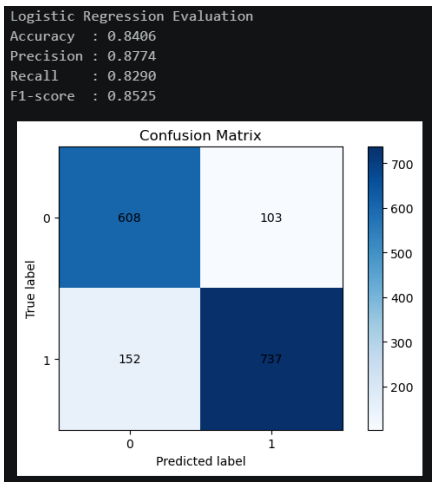
1. learning rate 0.1



2. learning rate 0.01

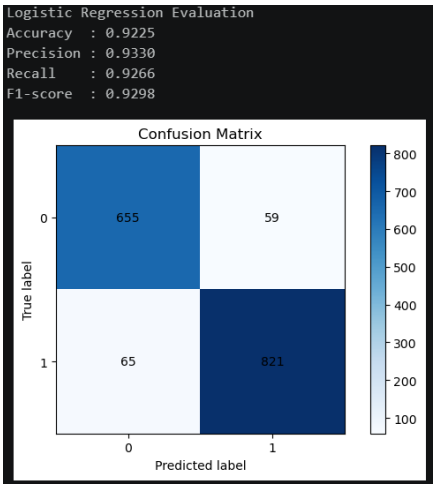


3. learning rate 0.001

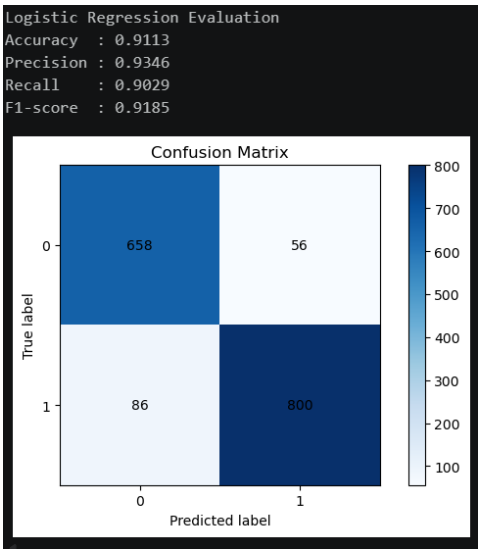


iv. logistic_data_D

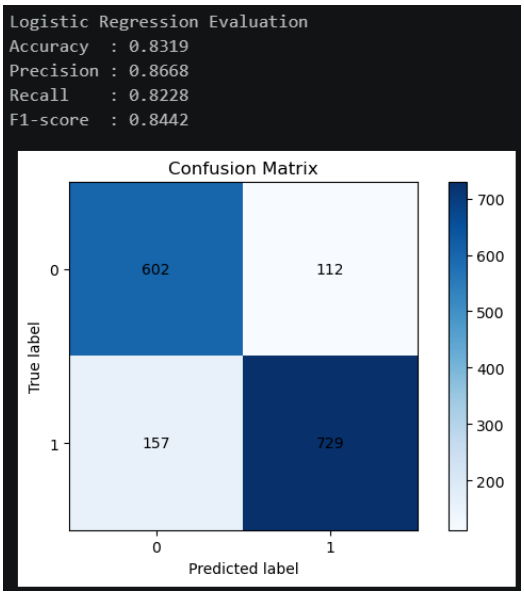
1. learning rate 0.1



2. learning rate 0.01



3. learning rate 0.001



c. **Describe your observations in 2(a) and 2(b).**

Optimal Convergence (lr = 0.1): Across all datasets, a learning rate of 0.1 provides the fastest and most stable convergence. The loss drops rapidly and flattens out well within the 500-iteration limit, leading to the highest overall classification metrics (Accuracy, Precision, Recall, and F1-score).

Slow but Steady Convergence (lr = 0.01): The loss curves show a smooth, gradual decrease but do not fully converge within 500 iterations. While the evaluation metrics are still highly acceptable, the model is clearly still learning and would likely achieve better performance with more training iterations.

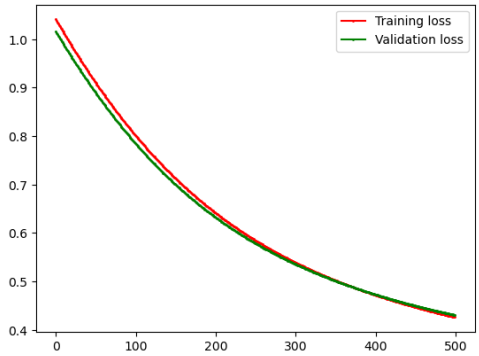
Underfitting (lr = 0.001): A learning rate of 0.001 is too small, causing the loss to decrease at an extremely slow rate. The model fails to converge within 500 iterations, resulting in the poorest classification performance and a significantly higher number of misclassifications in the confusion matrices.

d. **Change the number of training iterations to the following values, and plot the loss curve for each setting (using a fixed learning rate of 0.01):**

i. logistic_data_A

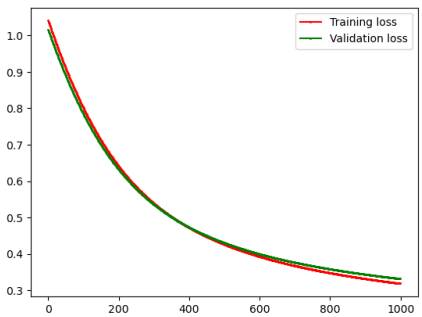
1. iteration 500

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
✓ 0.0s

50. Training loss: 0.9117697645477548, Val loss:0.8909436605779335
100. Training loss: 0.8023254524923707, Val loss:0.7857578532165119
150. Training loss: 0.7134127241759743, Val loss:0.700717653642279
200. Training loss: 0.6421019406042985, Val loss:0.6328939736163958
250. Training loss: 0.5851379734874116, Val loss:0.5790315939113776
300. Training loss: 0.5394700517290799, Val loss:0.536090659945112
350. Training loss: 0.5025287985878911, Val loss:0.5015278107602668
400. Training loss: 0.472286903656601, Val loss:0.4733528541640398
450. Training loss: 0.4472015131236836, Val loss:0.45006424591042526
500. Training loss: 0.42611745469430734, Val loss:0.43054684173876917
```



2. iteration 1000

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1000,val_ratio=0.2)
✓ 0.0s

50. Training loss: 0.9117697645477548, Val loss:0.8909436605779335
100. Training loss: 0.8023254524923707, Val loss:0.7857578532165119
150. Training loss: 0.7134127241759743, Val loss:0.700717653642279
200. Training loss: 0.6421019406042985, Val loss:0.6328939736163958
250. Training loss: 0.5851379734874116, Val loss:0.5790315939113776
300. Training loss: 0.5394700517290799, Val loss:0.536090659945112
350. Training loss: 0.5025287985878911, Val loss:0.5015278107602668
400. Training loss: 0.472286903656601, Val loss:0.4733528541640398
450. Training loss: 0.4472015131236836, Val loss:0.45006424591042526
500. Training loss: 0.42611745469430734, Val loss:0.43054684173876917
550. Training loss: 0.4081728908570478, Val loss:0.41397464715417137
600. Training loss: 0.392722162835905, Val loss:0.3997327089757893
650. Training loss: 0.37927745309555855, Val loss:0.38735889620900993
700. Training loss: 0.36746624563347524, Val loss:0.3765019336049602
750. Training loss: 0.3570007561624212, Val loss:0.36689152653138263
800. Training loss: 0.3476560265230133, Val loss:0.35831711935482174
850. Training loss: 0.3392541695963787, Val loss:0.3506127174310751
900. Training loss: 0.33165295481794593, Val loss:0.34364595163394807
950. Training loss: 0.3247374619252417, Val loss:0.3373101204935873
1000. Training loss: 0.31841391608892583, Val loss:0.3315183366092 56
```

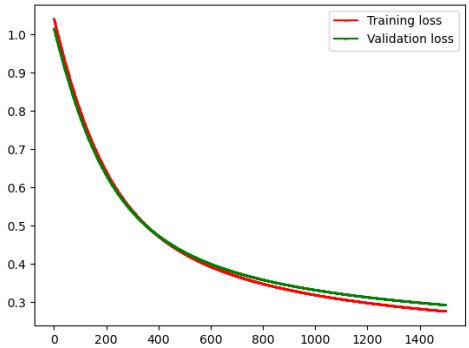


3. iteration 1500

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1500,val_ratio=0.2)
```

✓ 0.1s

```
50. Training loss: 0.9117697645477548, Val loss:0.8909436605779335
100. Training loss: 0.8023254524923707, Val loss:0.7857578532165119
150. Training loss: 0.7134127241759743, Val loss:0.700717653642279
200. Training loss: 0.6421019406042985, Val loss:0.6328939736163958
250. Training loss: 0.5851379734874116, Val loss:0.5790315939113776
300. Training loss: 0.5394700517290799, Val loss:0.536090659945112
350. Training loss: 0.5025287985878911, Val loss:0.5015278107602668
400. Training loss: 0.472286903656601, Val loss:0.4733528541640398
450. Training loss: 0.4472015131236836, Val loss:0.45006424591042526
500. Training loss: 0.42611745469430734, Val loss:0.43054684173876917
550. Training loss: 0.4081728908570478, Val loss:0.41397464715417137
600. Training loss: 0.392722162835905, Val loss:0.3997327089757893
650. Training loss: 0.37927745309555855, Val loss:0.38735889620900993
700. Training loss: 0.36746624563347524, Val loss:0.3765019336049602
750. Training loss: 0.3570007561624212, Val loss:0.36689152653138263
800. Training loss: 0.3476560265230133, Val loss:0.35831711935482174
850. Training loss: 0.3392541695963787, Val loss:0.3506127174310751
900. Training loss: 0.33165295481794593, Val loss:0.34364595163394807
950. Training loss: 0.3247374619252417, Val loss:0.3373101204935873
1000. Training loss: 0.31841391608892583, Val loss:0.331518336609256
1050. Training loss: 0.3126050864623 31, Val loss:0.32619917348528593
1100. Training loss: 0.30724681566915024, Val loss:0.32129339291248765
1150. Training loss: 0.30228537544269035, Val loss:0.3167514586137984
1200. Training loss: 0.29767543180663514, Val loss:0.31253162800098494
1250. Training loss: 0.29337846443896937, Val loss:0.308598473367542
...
1350. Training loss: 0.28559627134081733, Val loss:0.30147535992311736
1400. Training loss: 0.28205815951999796, Val loss:0.29823687322865977
1450. Training loss: 0.27872584414203677, Val loss:0.2951866985770722
1500. Training loss: 0.27558065740555393, Val loss:0.2923077366055479
```



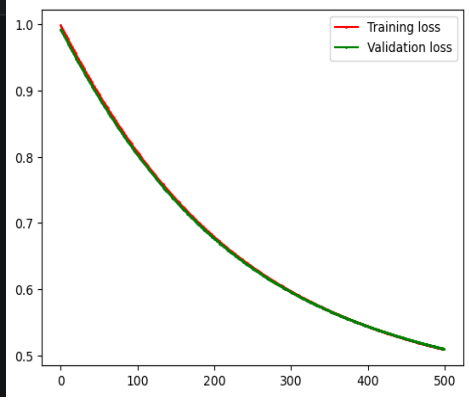
ii. logistic_data_B

1. iteration 500

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.0s

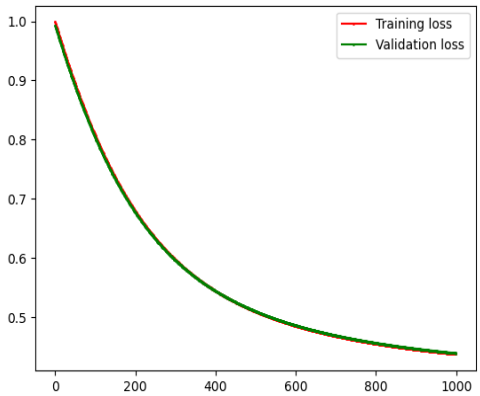
```
50. Training loss: 0.8957410640363592, Val loss:0.8906374105058227
100. Training loss: 0.8083096467352318, Val loss:0.804210107205688
150. Training loss: 0.7369603264945955, Val loss:0.7337661725077593
200. Training loss: 0.6795500174801885, Val loss:0.6771732291658011
250. Training loss: 0.6336948605195072, Val loss:0.6320453148336889
300. Training loss: 0.5971080713785597, Val loss:0.5960905942359478
350. Training loss: 0.5678011822962918, Val loss:0.567319509722204
400. Training loss: 0.5441528833412146, Val loss:0.5441156176252377
450. Training loss: 0.5248927073071165, Val loss:0.5252178109592448
500. Training loss: 0.5090467133965937, Val loss:0.5096635682600803
```



2. iteration 1000

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1000,val_ratio=0.2)
```
✓ 0.1s

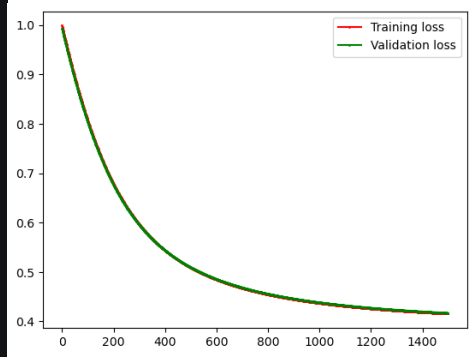
```
50. Training loss: 0.8957410640363592, Val loss:0.8906374105058227
100. Training loss: 0.8083096467352318, Val loss:0.804210107205688
150. Training loss: 0.7369603264945955, Val loss:0.7337661725077593
200. Training loss: 0.6795500174801885, Val loss:0.6771732291658011
250. Training loss: 0.6336948605195072, Val loss:0.6320453148336889
300. Training loss: 0.5971080713785597, Val loss:0.5960905942359478
350. Training loss: 0.5678011822962918, Val loss:0.567319509722204
400. Training loss: 0.5441528833412146, Val loss:0.5441156176252377
450. Training loss: 0.5248927073071165, Val loss:0.5252178109592448
500. Training loss: 0.5090467133965937, Val loss:0.5096635682600803
550. Training loss: 0.49587540253738227, Val loss:0.4967249285603095
600. Training loss: 0.4848180760479444, Val loss:0.4858517431539401
650. Training loss: 0.47544801408599946, Val loss:0.47662640141947243
700. Training loss: 0.46743824703384007, Val loss:0.4687295376448458
750. Training loss: 0.4605361029338294, Val loss:0.4619147020729587
800. Training loss: 0.45454450065677376, Val loss:0.4559898473239186
850. Training loss: 0.44930824166600186, Val loss:0.45080382236615935
900. Training loss: 0.44470394251514456, Val loss:0.44623648933701326
950. Training loss: 0.4406326021036907, Val loss:0.4421914481037607
1000. Training loss: 0.4370140754178422, Val loss:0.4385906395654831
```



3. iteration 1500

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1500,val_ratio=0.2)
```
✓ 0.1s

```
50. Training loss: 0.8957410640363592, Val loss:0.8906374105058227
100. Training loss: 0.8083096467352318, Val loss:0.804210107205688
150. Training loss: 0.7369603264945955, Val loss:0.7337661725077593
200. Training loss: 0.6795500174801885, Val loss:0.6771732291658011
250. Training loss: 0.6336948605195072, Val loss:0.6320453148336889
300. Training loss: 0.5971080713785597, Val loss:0.5960905942359478
350. Training loss: 0.5678011822962918, Val loss:0.567319509722204
400. Training loss: 0.5441528833412146, Val loss:0.5441156176252377
450. Training loss: 0.5248927073071165, Val loss:0.5252178109592448
500. Training loss: 0.5090467133965937, Val loss:0.5096635682600803
550. Training loss: 0.49587540253738227, Val loss:0.4967249285603095
600. Training loss: 0.4848180760479444, Val loss:0.4858517431539401
650. Training loss: 0.47544801408599946, Val loss:0.47662640141947243
700. Training loss: 0.46743824703384007, Val loss:0.4687295376448458
750. Training loss: 0.4605361029338294, Val loss:0.4619147020729587
800. Training loss: 0.45454450065677376, Val loss:0.4559898473239186
850. Training loss: 0.44930824166600186, Val loss:0.45080382236615935
900. Training loss: 0.44470394251514456, Val loss:0.44623648933701326
950. Training loss: 0.4406326021036907, Val loss:0.4421914481037607
1000. Training loss: 0.4370140754178422, Val loss:0.4385906395654831
1050. Training loss: 0.4337829317564017, Val loss:0.43537030847184505
1100. Training loss: 0.4308853242253363, Val loss:0.4324779564105105
1150. Training loss: 0.42827660314957516, Val loss:0.42987002147557235
1200. Training loss: 0.42591948100889654, Val loss:0.42751009564795883
1250. Training loss: 0.42378260959133457, Val loss:0.4253675434444254
...
1350. Training loss: 0.420067485328447, Val loss:0.4216346337770525
1400. Training loss: 0.4184473473654607, Val loss:0.4200032461158787
1450. Training loss: 0.4169624383563537, Val loss:0.41850595745596986
1500. Training loss: 0.41559839451552544, Val loss:0.4171286601466142
```



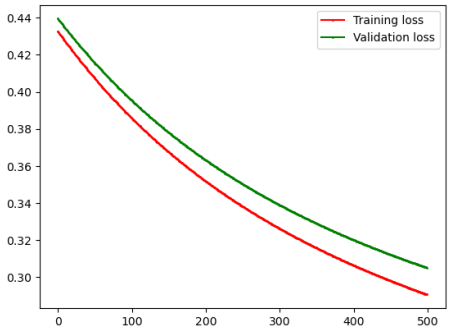
iii. logistic_data_C

1. iteration 500

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.1s

```
50. Training loss: 0.40763335808873047, Val loss:0.41596249839320293
100. Training loss: 0.3860913696555631, Val loss:0.3956206679931872
150. Training loss: 0.3677333549357604, Val loss:0.3782638648505192
200. Training loss: 0.35195423947335824, Val loss:0.36332831682999583
250. Training loss: 0.33827534357025274, Val loss:0.3503673663467325
300. Training loss: 0.32631769333124555, Val loss:0.3390266190729544
350. Training loss: 0.31578039047898315, Val loss:0.32902400827673756
400. Training loss: 0.3064235064986363, Val loss:0.3201340925514732
450. Training loss: 0.29805473954887807, Val loss:0.3121758074752029
500. Training loss: 0.2905190785492918, Val loss:0.3050029544730205
```

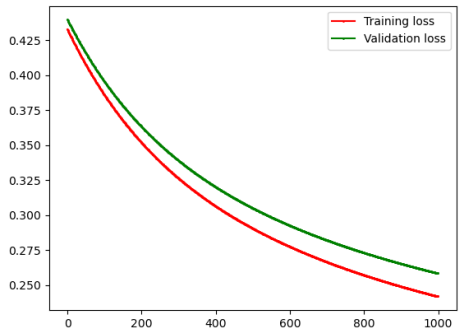


2. iteration 1000

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1000,val_ratio=0.2)
```

✓ 0.1s

```
50. Training loss: 0.40763335808873047, Val loss:0.41596249839320293
100. Training loss: 0.3860913696555631, Val loss:0.3956206679931872
150. Training loss: 0.3677333549357604, Val loss:0.3782638648505192
200. Training loss: 0.35195423947335824, Val loss:0.36332831682999583
250. Training loss: 0.33827534357025274, Val loss:0.3503673663467325
300. Training loss: 0.32631769333124555, Val loss:0.3390266190729544
350. Training loss: 0.31578039047898315, Val loss:0.32902400827673756
400. Training loss: 0.3064235064986363, Val loss:0.3201340925514732
450. Training loss: 0.29805473954887807, Val loss:0.3121758074752029
500. Training loss: 0.2905190785492918, Val loss:0.3050029544730205
550. Training loss: 0.28369081715291194, Val loss:0.29849682447858894
600. Training loss: 0.2774673815390825, Val loss:0.2925604725198355
650. Training loss: 0.27176454986406984, Val loss:0.2871142640230998
700. Training loss: 0.26651273728471975, Val loss:0.28209239966841937
750. Training loss: 0.2616540972661098, Val loss:0.2774401937722723
800. Training loss: 0.2571402495104391, Val loss:0.2731119340530531
850. Training loss: 0.2529304904135508, Val loss:0.2690691912053621
900. Training loss: 0.24899037649704744, Val loss:0.2652794776487482
950. Training loss: 0.24529059735148753, Val loss:0.2617151783447419
1000. Training loss: 0.2418060743124167, Val loss:0.25835269445470743
```

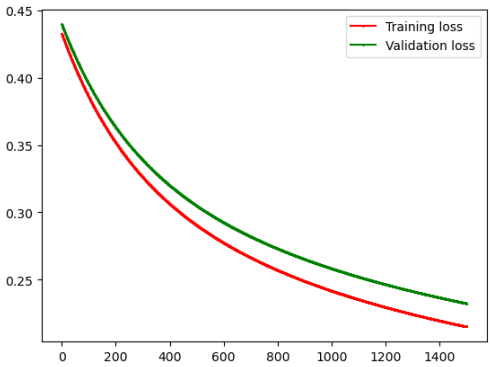


3. iteration 1500

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1500,val_ratio=0.2)
```

✓ 0.1s

```
50. Training loss: 0.40763335808873047, Val loss:0.41596249839320293
100. Training loss: 0.3860913696555631, Val loss:0.3956206679931872
150. Training loss: 0.3677333549357604, Val loss:0.3782638648505192
200. Training loss: 0.35195423947335824, Val loss:0.36332831682999583
250. Training loss: 0.33827534357025274, Val loss:0.3503673663467325
300. Training loss: 0.32631769333124555, Val loss:0.3390266190729544
350. Training loss: 0.31578039047898315, Val loss:0.32902400827673756
400. Training loss: 0.3064235064986363, Val loss:0.3201340925514732
450. Training loss: 0.29805473954887807, Val loss:0.3121758074752029
500. Training loss: 0.2905190785492918, Val loss:0.3050029544730205
550. Training loss: 0.28369081715291194, Val loss:0.29849682447858894
600. Training loss: 0.2774673815390825, Val loss:0.2925604725198355
650. Training loss: 0.27176454986406984, Val loss:0.2871142640230998
700. Training loss: 0.26651273728471975, Val loss:0.28209239966841937
750. Training loss: 0.2616540972661098, Val loss:0.2774401937722723
800. Training loss: 0.2571402495104391, Val loss:0.2731119340530531
850. Training loss: 0.2529304904135508, Val loss:0.2690691912053621
900. Training loss: 0.24899037649704744, Val loss:0.2652794776487482
950. Training loss: 0.24529059735148753, Val loss:0.2617151783447419
1000. Training loss: 0.2418060743124167, Val loss:0.25835269445470743
1050. Training loss: 0.2385152359607014, Val loss:0.2551717542053155
1100. Training loss: 0.23539943279363662, Val loss:0.2521548556814715
1150. Training loss: 0.23244246195655585, Val loss:0.24928681416920329
1200. Training loss: 0.22963017943129088, Val loss:0.24655439272017024
1250. Training loss: 0.22695018205173692, Val loss:0.2439459992546035
...
1350. Training loss: 0.22194460765068139, Val loss:0.23906169862217547
1400. Training loss: 0.21960078795394633, Val loss:0.23676879378606558
1450. Training loss: 0.2173524372241134, Val loss:0.23456560709560623
1500. Training loss: 0.21519271119800445, Val loss:0.232445777751773
```



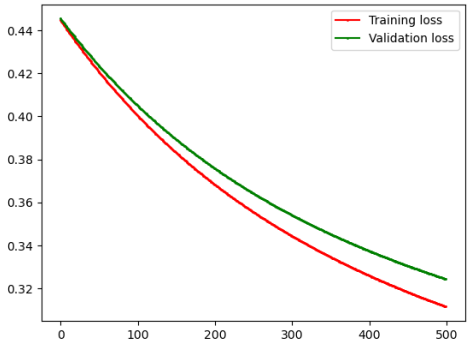
iv. logistic_data_D

1. iteration 500

```python
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

✓ 0.1s

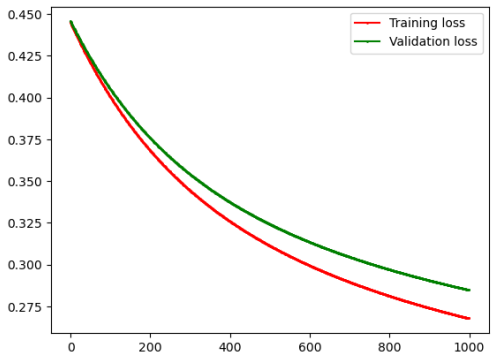
```
50. Training loss: 0.4210863364107005, Val loss:0.42389318391426317
100. Training loss: 0.4006650658061349, Val loss:0.40526939422934116
150. Training loss: 0.3833104146387553, Val loss:0.3894644471413472
200. Training loss: 0.36844391326627834, Val loss:0.37594512076571973
250. Training loss: 0.3556059025863166, Val loss:0.3642876353134937
300. Training loss: 0.3444310989408412, Val loss:0.354154961784246
350. Training loss: 0.33462857869617535, Val loss:0.3452784421002676
400. Training loss: 0.32596580182229623, Val loss:0.33744318725215144
450. Training loss: 0.3182560482209092, Val loss:0.33047660104180177
500. Training loss: 0.3113486162459987, Val loss:0.3242394100832966
```



2. iteration 1000

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1000,val_ratio=0.2)
```
✓ 0.1s

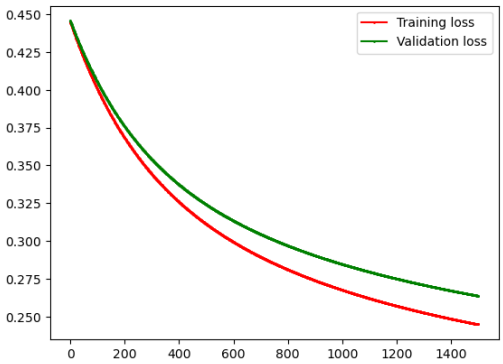
```
50. Training loss: 0.4210863364107005, Val loss:0.42389318391426317
100. Training loss: 0.4006650658061349, Val loss:0.40526939422934116
150. Training loss: 0.3833104146387553, Val loss:0.3894644471413472
200. Training loss: 0.36844391326627834, Val loss:0.37594512076571973
250. Training loss: 0.3556059025863166, Val loss:0.3642876353134937
300. Training loss: 0.3444310989408412, Val loss:0.354154961784246
350. Training loss: 0.33462857869617535, Val loss:0.3452784421002676
400. Training loss: 0.32596580182229623, Val loss:0.33744318725215144
450. Training loss: 0.3182560482209092, Val loss:0.33047660104180177
500. Training loss: 0.3113486162459987, Val loss:0.3242394100832966
550. Training loss: 0.30512120261559816, Val loss:0.31861866833458574
600. Training loss: 0.2994739813818718, Val loss:0.31352230201213016
650. Training loss: 0.29432499674933427, Val loss:0.3088748500410424
700. Training loss: 0.28960656875428886, Val loss:0.3046141303741866
750. Training loss: 0.2852624794460229, Val loss:0.300688623158738
800. Training loss: 0.2812457612926405, Val loss:0.2970554094989434
850. Training loss: 0.27751695137408533, Val loss:0.29367854168015783
900. Training loss: 0.27404270696942545, Val loss:0.2905277493299533
950. Training loss: 0.27079470256395355, Val loss:0.2875774079495663
1000. Training loss: 0.2677487468716707, Val loss:0.28480571306479263
```



3. iteration 1500

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =1500,val_ratio=0.2)
```
✓ 0.1s

```
50. Training loss: 0.4210863364107005, Val loss:0.42389318391426317
100. Training loss: 0.4006650658061349, Val loss:0.40526939422934116
150. Training loss: 0.3833104146387553, Val loss:0.3894644471413472
200. Training loss: 0.36844391326627834, Val loss:0.37594512076571973
250. Training loss: 0.3556059025863166, Val loss:0.3642876353134937
300. Training loss: 0.3444310989408412, Val loss:0.354154961784246
350. Training loss: 0.33462857869617535, Val loss:0.3452784421002676
400. Training loss: 0.32596580182229623, Val loss:0.33744318725215144
450. Training loss: 0.3182560482209092, Val loss:0.33047660104180177
500. Training loss: 0.3113486162459987, Val loss:0.3242394100832966
550. Training loss: 0.30512120261559816, Val loss:0.31861866833458574
600. Training loss: 0.2994739813818718, Val loss:0.31352230201213016
650. Training loss: 0.29432499674933427, Val loss:0.3088748500410424
700. Training loss: 0.28960656875428886, Val loss:0.3046141303741866
750. Training loss: 0.2852624794460229, Val loss:0.300688623158738
800. Training loss: 0.2812457612926405, Val loss:0.2970554094989434
850. Training loss: 0.27751695137408533, Val loss:0.29367854168015783
900. Training loss: 0.27404270696942545, Val loss:0.2905277493299533
950. Training loss: 0.27079470256395355, Val loss:0.2875774079495663
1000. Training loss: 0.2677487468716707, Val loss:0.28480571306479263
1050. Training loss: 0.2648840725900826, Val loss:0.28219401611654
1100. Training loss: 0.2621827623556451, Val loss:0.279726288068611
1150. Training loss: 0.2596292825708435, Val loss:0.27738868427016106
1200. Training loss: 0.25721010304879466, Val loss:0.2751691899199601
1250. Training loss: 0.2549133852364442, Val loss:0.27305732995554827
...
1350. Training loss: 0.25064694270321475, Val loss:0.2691209228699491
1400. Training loss: 0.24865990194688906, Val loss:0.2672811790675561
1450. Training loss: 0.246760367239873, Val loss:0.2655183776381135
1500. Training loss: 0.24494187826257335, Val loss:0.26382688963501755
```

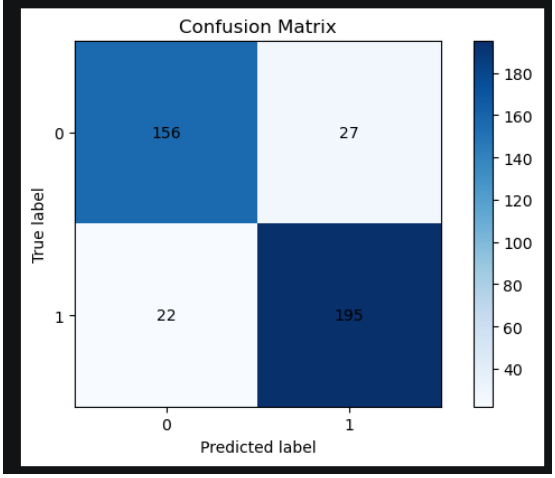


e. **Following 2(d), show the evaluations by using the "evaluate_binary_classifier" function.**

i. logistic_data_A

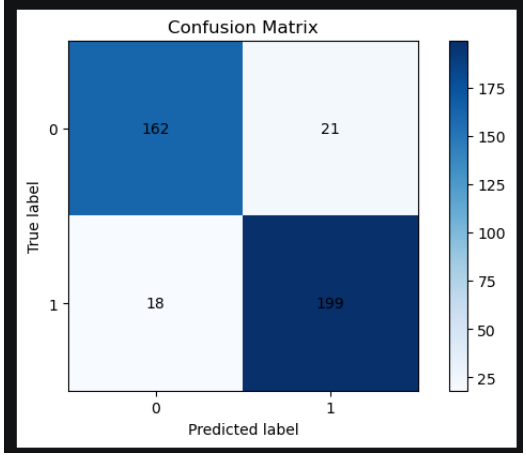
1. iteration 500

```
Logistic Regression Evaluation
Accuracy  : 0.8775
Precision : 0.8784
Recall    : 0.8986
F1-score  : 0.8884
```



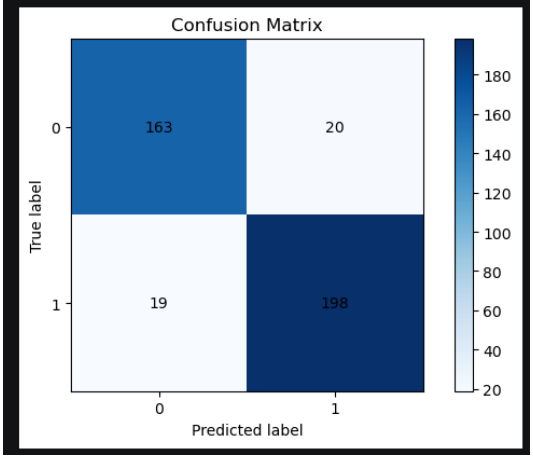
2. iteration 1000

```
Logistic Regression Evaluation
Accuracy  : 0.9025
Precision : 0.9045
Recall    : 0.9171
F1-score  : 0.9108
```



3. iteration 1500

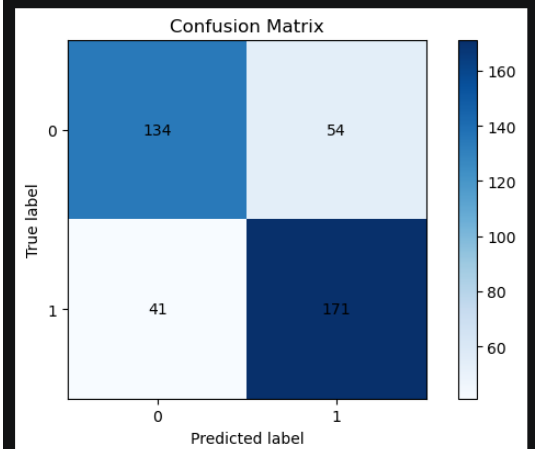
```
Logistic Regression Evaluation
Accuracy  : 0.9025
Precision : 0.9083
Recall    : 0.9124
F1-score  : 0.9103
```



ii. logistic_data_B

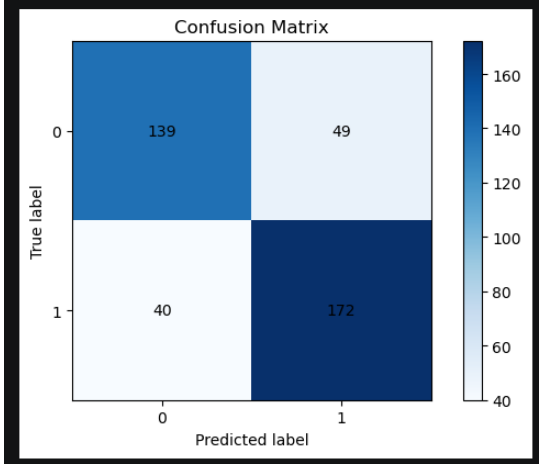
1. iteration 500

```
Logistic Regression Evaluation
Accuracy  : 0.7625
Precision : 0.7600
Recall    : 0.8066
F1-score  : 0.7826
```



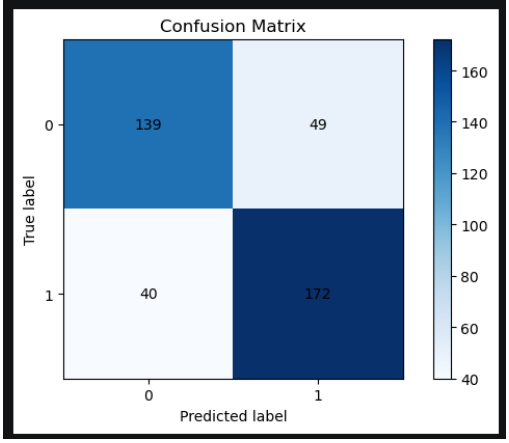
2. iteration 1000

```
Logistic Regression Evaluation
Accuracy  : 0.7775
Precision : 0.7783
Recall    : 0.8113
F1-score  : 0.7945
```



3. iteration 1500

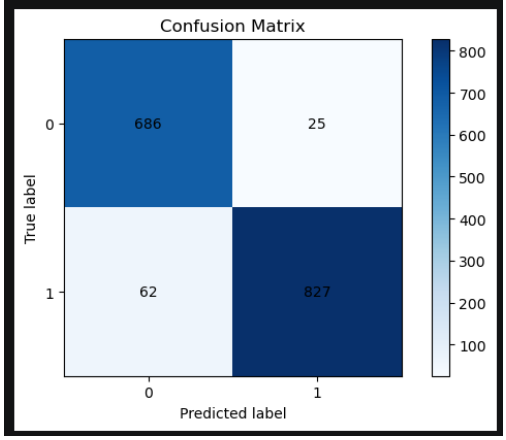
```
Logistic Regression Evaluation
Accuracy  : 0.7775
Precision : 0.7783
Recall    : 0.8113
F1-score  : 0.7945
```



iii. logistic_data_C

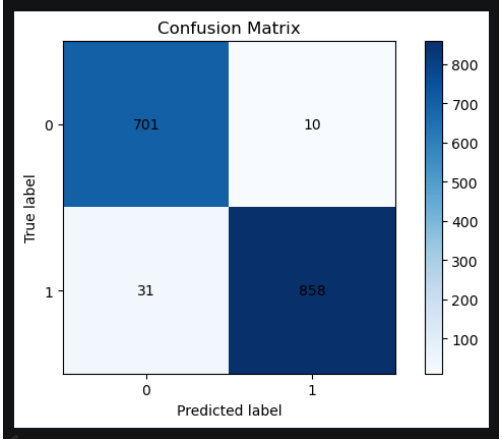
1. iteration 500

```
Logistic Regression Evaluation
Accuracy  : 0.9456
Precision : 0.9707
Recall    : 0.9303
F1-score  : 0.9500
```



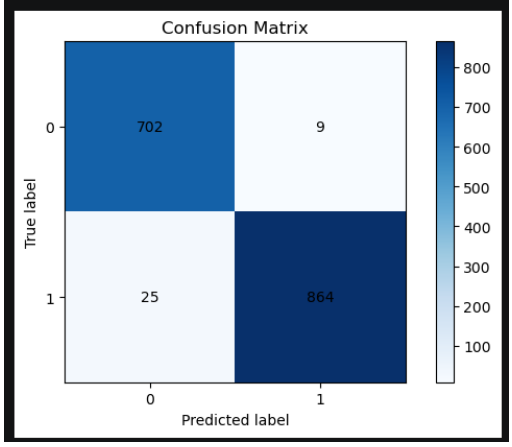
2. iteration 1000

```
Logistic Regression Evaluation
Accuracy  : 0.9744
Precision : 0.9885
Recall    : 0.9651
F1-score  : 0.9767
```



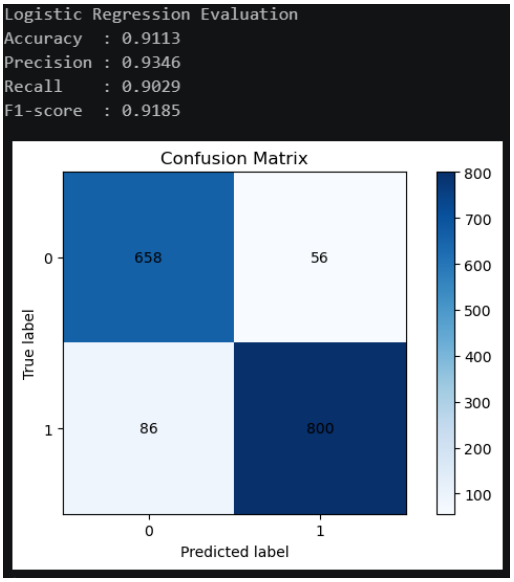
3. iteration 1500

```
Logistic Regression Evaluation
Accuracy  : 0.9788
Precision : 0.9897
Recall    : 0.9719
F1-score  : 0.9807
```

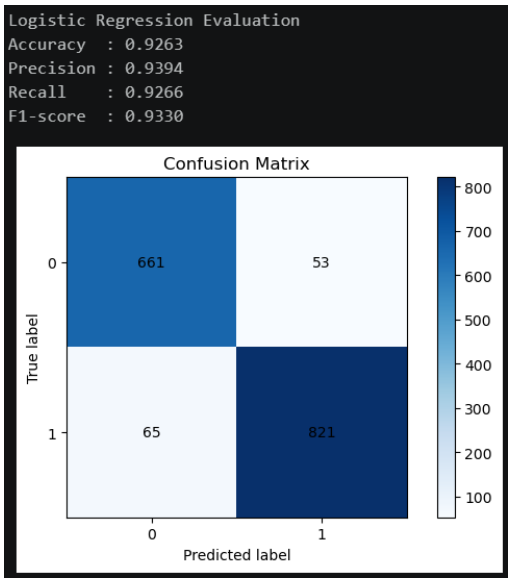


iv. logistic_data_D

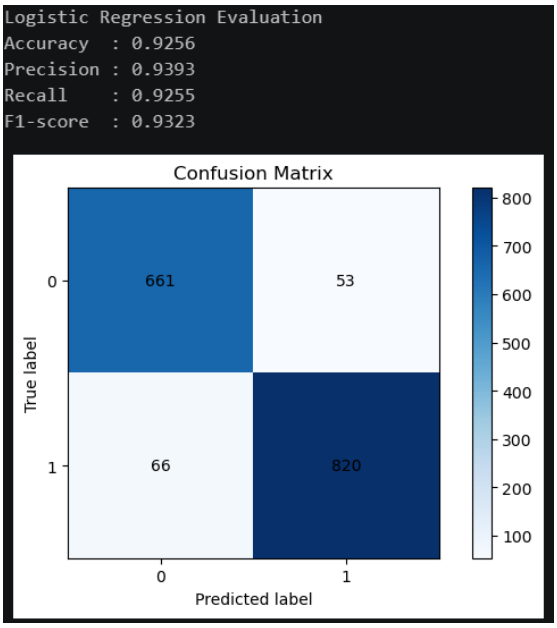
1. iteration 500



2. iteration 1000



3. iteration 1500



f. **Describe your observations in 2(d) and 2(e).**

Initial Improvement (Solving Underfitting): Increasing the iterations from 500 to 1000 significantly improves all evaluation metrics (Accuracy, Precision, Recall, F1-score) and reduces the loss. This confirms that at 500 iterations, the model had not yet fully converged and needed more training epochs to reach its potential.

Convergence and Diminishing Returns: When comparing 1000 iterations to 1500 iterations, the performance plateaus. For example, in Dataset A, the accuracy remains strictly identical at 0.9025. This indicates that the model successfully converges around 1000 iterations, and adding more training epochs yields no meaningful benefit.

Risk of Overfitting: In Dataset D, pushing the training to 1500 iterations actually results in a slight decrease in overall performance (Accuracy drops marginally from 0.9263 to 0.9256) compared to 1000 iterations. This illustrates the early signs of overfitting, where training the model too long after convergence causes it to memorize noise in the training data rather than generalizing well to unseen data.

3. Data Preprocessing (20 points):

a. **Report the median and standard deviation for columns with missing values.**

Column	Median	Standard Deviation
SepalLengthCm	6.3000	1.0371
SepalWidthCm	2.9000	0.3896
PetalLengthCm	5.0856	1.5828
PetalWidthCm	1.6000	0.7067
BranchLength	16.3000	1.0352

b. **Write the code to replace the missing values using "Nearest Neighbors Imputation" for columns with missing**

values. Report the median and standard deviation of those columns. Compare your result with 3(a) and explain your observation.

Column	Median (After)	Standard Deviation (After)
SepalLengthCm	6.3000	1.0336
SepalWidthCm	3.0000	0.4439
PetalLengthCm	5.1000	1.5815
PetalWidthCm	1.6000	0.7061
BranchLength	16.3000	1.0332

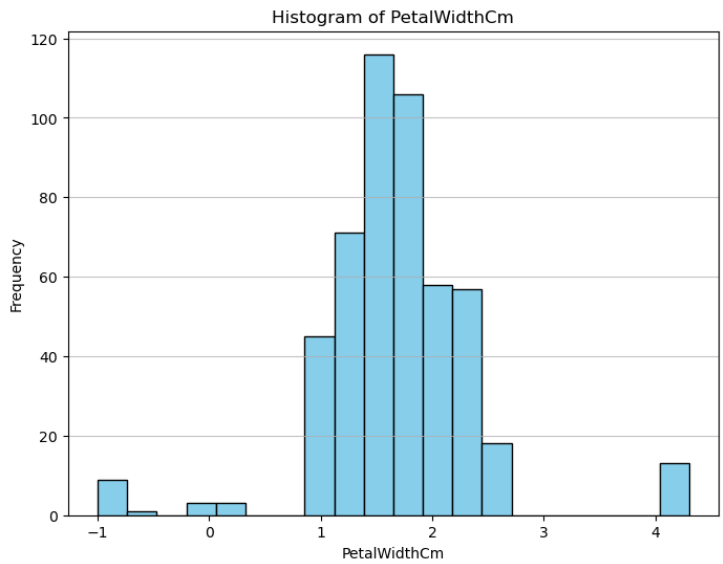
Comparing the statistics before and after the Nearest Neighbors Imputation, the medians for SepalLengthCm, PetalWidthCm, and BranchLength remained unchanged. However, the median of SepalWidthCm shifted from 2.9000 to 3.0000, and PetalLengthCm slightly changed from 5.0856 to 5.1000. Additionally, the standard deviations for almost all columns slightly decreased, with the notable exception of SepalWidthCm, which increased from 0.3896 to 0.4439.

Because the imputed values are local averages, they tend to slightly reduce the overall variance and standard deviation of the features. The noticeable shift in the median and the increase in the standard

deviation for SepalWidthCm suggest that the specific data points with missing values belonged to instances whose nearest neighbors had systematically higher and more varied values for that feature.

4. Data Exploration (20 points):

a. **Plot the histogram of the column "PetalWidthCm" in "NYCU_Iris.csv". Please describe your observations for the histogram.**



The majority of the PetalWidthCm data is highly concentrated between 1.0 and 2.5, forming a clear unimodal distribution with a peak around 1.5.

There are noticeable outliers on both extremes of the distribution. This includes some physically impossible negative values (around -1.0) and a small cluster of extremely high values (around 4.0), indicating potential noise or data entry errors in the dataset.

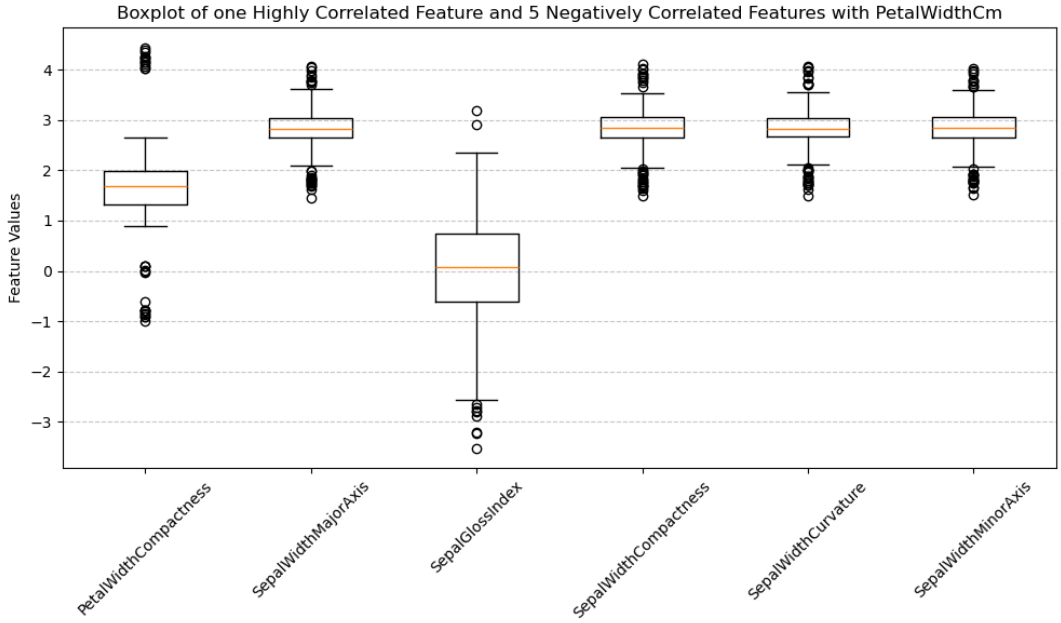
b. **List the feature (excluding “PetalWidthCm”, “Species”, and “Id”) that has the largest positive correlation (Pearson correlation coefficient) with the “PetalWidthCm” column.**

The feature with the largest positive correlation to PetalWidthCm is PetalWidthCompactness, with a Pearson correlation coefficient of 0.9917.

c. **Report the top 5 features with the strongest negative correlations with the 'PetalWidthCm' column.**

1.SepalWidthMajorAxis (Pearson correlation: -0.0964)
2.SepalGlossIndex (Pearson correlation: -0.0952)
3.SepalWidthCompactness (Pearson correlation: -0.0885)
4.SepalWidthCurvature (Pearson correlation: -0.0813)
5.SepalWidthMinorAxis (Pearson correlation: -0.0744)

d. **Plot the boxplot for all features found in 4(b) and 4(c).**

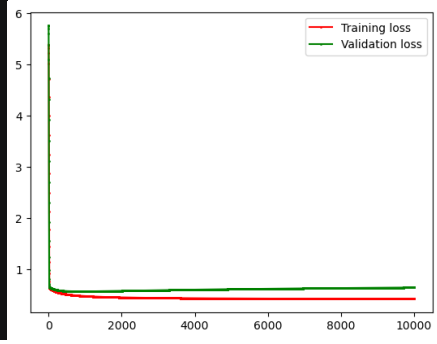


5. Regularization (20 points):

a. **L2 Regularization. Please run Real_World_Classification.ipynb to compare the performance between no regularization (current code) and L2 regularization. For L2 regularization, set the regularization parameter (reg_lambda in the code) to 0.01, 1, and 100, respectively. Then, plot the loss curves for all settings:**

i. No regularization

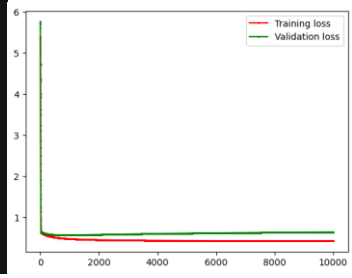
```
50. Training loss: 0.6183886535310102, Val loss:0.638338191177124
100. Training loss: 0.5960033921480783, Val loss:0.6229857283892484
150. Training loss: 0.5781690886526527, Val loss:0.611179684417757
200. Training loss: 0.5636439627476497, Val loss:0.601978582536059
250. Training loss: 0.5515861130769195, Val loss:0.5947349187857767
300. Training loss: 0.5414115554176107, Val loss:0.588991764914238
350. Training loss: 0.5327055446405081, Val loss:0.5844187428740432
400. Training loss: 0.5251667000245236, Val loss:0.5807716266567299
450. Training loss: 0.5185712433977272, Val loss:0.5778662753932695
500. Training loss: 0.5127496761323889, Val loss:0.5755614147648992
550. Training loss: 0.5075712624849853, Val loss:0.573746996512145
600. Training loss: 0.5029334978289509, Val loss:0.5723361523089563
650. Training loss: 0.49875481918863035, Val loss:0.571259514930716
700. Training loss: 0.4949694641429734, Val loss:0.5704611319860385
750. Training loss: 0.49152377948438897, Val loss:0.569895472931603
800. Training loss: 0.48837352569502923, Val loss:0.5695252010832258
850. Training loss: 0.4854818772128884, Val loss:0.5693194905552285
900. Training loss: 0.4828179168605863, Val loss:0.569252737858429
950. Training loss: 0.4803554867354189, Val loss:0.5693035637376979
1000. Training loss: 0.4780723000420191, Val loss:0.5694540314846964
1050. Training loss: 0.4759492466098327, Val loss:0.5696890288047686
1100. Training loss: 0.4739698440522338, Val loss:0.5699957747186661
1150. Training loss: 0.4721197997758728, Val loss:0.5703634230835796
1200. Training loss: 0.4703866583152128, Val loss:0.5707827415094618
1250. Training loss: 0.4687595150335521, Val loss:0.57124584963531
...
9850. Training loss: 0.4224944210043737, Val loss:0.6449216457898543
9900. Training loss: 0.4224312146326115, Val loss:0.6451644818670514
9950. Training loss: 0.42236835747022444, Val loss:0.645406035688366
10000. Training loss: 0.4223058442592935, Val loss:0.6456463161358968
```



ii. L2 (reg_lambda=0.01)

```
model = LinearModel(dim=X_train.shape[1], is_reg=False, loss_fn=loss_fn, act_fn=act_fn, grad_fn=grad_fn)
model.fit(X_train, y_train,lr=0.1, n_iteration =10000,val_ratio=0.2, reg_type='l2', reg_lambda=0.01)

# print model parameters
print("Model parameters (weights):", model.W)
# sum of absolute values of weights
print("Sum of absolute values of weights:", np.sum(np.abs(model.W)))
✓ 0.4s
```

```
50. Training loss: 0.6183968442308057, Val loss:0.6383418749174559
100. Training loss: 0.5960218584459506, Val loss:0.6229937284988916
150. Training loss: 0.5781987940557217, Val loss:0.6111917862872271
200. Training loss: 0.563685266911725, Val loss:0.6019940821319419
250. Training loss: 0.5516390786456719, Val loss:0.5947528491389577
300. Training loss: 0.5414760921827046, Val loss:0.5890110215795246
350. Training loss: 0.5327814781753895, Val loss:0.5844381571451082
400. Training loss: 0.5252538055110114, Val loss:0.5807900103412457
450. Training loss: 0.5186692635109195, Val loss:0.5778824491950175
500. Training loss: 0.5128583316269498, Val loss:0.5755742266452543
550. Training loss: 0.5076902593666242, Val loss:0.57375533358103
600. Training loss: 0.5030625328026356, Val loss:0.5723389483402411
650. Training loss: 0.4988935840888995, Val loss:0.5712557548145529
700. Training loss: 0.4951176496280415, Val loss:0.5704498539888323
750. Training loss: 0.4916810781087383, Val loss:0.5698757693706247
800. Training loss: 0.4885396344199364, Val loss:0.5694962178557675
850. Training loss: 0.4856564994033308, Val loss:0.5692804258497898
900. Training loss: 0.48300076382002083, Val loss:0.5692028402956646
950. Training loss: 0.4805462788346178, Val loss:0.5692421301298547
1000. Training loss: 0.4782707675002541, Val loss:0.5693804043573255
1050. Training loss: 0.4761551299855625, Val loss:0.5696025937896587
1100. Training loss: 0.4741828944966999, Val loss:0.5698959578997027
1150. Training loss: 0.47233977909761626, Val loss:0.5702496883559051
1200. Training loss: 0.4706133388973484, Val loss:0.5706545879956382
1250. Training loss: 0.468992679639337 94, Val loss:0.5711028091892127
...
9850. Training loss: 0.4231571061318596, Val loss:0.6406343773246127
9900. Training loss: 0.4230961248746708, Val loss:0.6408511575772493
9950. Training loss: 0.4230355003243262 4, Val loss:0.6410666695749779
10000. Training loss: 0.42297522725510195, Val loss:0.64128092256715
```

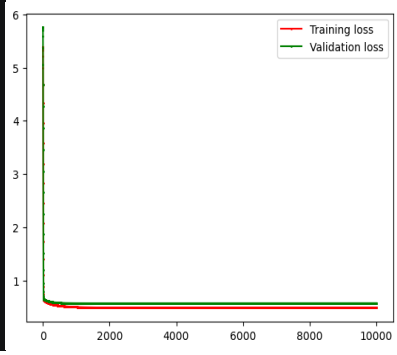


iii. L2 (reg_lambda=1)

```python
model = LinearModel(dim=X_train.shape[1], is_reg=False, loss_fn=loss_fn, act_fn=act_fn, grad_fn=grad_fn)
model.fit(X_train, y_train,lr=0.1, n_iteration =10000,val_ratio=0.2, reg_type='l2', reg_lambda=1)

# print model parameters
print("Model parameters (weights):", model.W)
# sum of absolute values of weights
print("Sum of absolute values of weights:", np.sum(np.abs(model.W)))
```

✓ 0.4s

```
50. Training loss: 0.6192229675550639, Val loss:0.6387339171797571
100. Training loss: 0.5979091328910923, Val loss:0.6238920902494233
150. Training loss: 0.5812650420168968, Val loss:0.6126225576568589
200. Training loss: 0.5679827993268195, Val loss:0.6039318894908268
250. Training loss: 0.5571858255876309, Val loss:0.5971444430979668
300. Training loss: 0.5482708567526591, Val loss:0.59178857989705
350. Training loss: 0.5408117331839131, Val loss:0.5875271116871134
400. Training loss: 0.5344999386168846, Val loss:0.5841138047798571
450. Training loss: 0.5291070807888374, Val loss:0.5813654836364932
500. Training loss: 0.524460662779445, Val loss:0.5791436781671647
550. Training loss: 0.520428062804602, Val loss:0.5773422515546591
600. Training loss: 0.5169056942810414, Val loss:0.5758788683267105
650. Training loss: 0.5138115085968704, Val loss:0.57468898760961
700. Training loss: 0.5110797027848409, Val loss:0.5737215545999529
750. Training loss: 0.5086569127317132, Val loss:0.5729358582350903
800. Training loss: 0.5064994275440443, Val loss:0.572299205209288
850. Training loss: 0.504571119143752, Val loss:0.5717851754632775
900. Training loss: 0.502841881584958, Val loss:0.5713722983963053
950. Training loss: 0.5012864394658483, Val loss:0.5710430378063318
1000. Training loss: 0.4998834275259014, Val loss:0.5707830062537872
1050. Training loss: 0.4986146721536377, Val loss:0.5705803518566839
1100. Training loss: 0.49746462505638617, Val loss:0.5704252760083235
1150. Training loss: 0.4964199128747678, Val loss:0.5703096514183474
1200. Training loss: 0.49546897604324036, Val loss:0.5702267176725083
1250. Training loss: 0.49460177698798324, Val loss:0.5701708371468824
...
9850. Training loss: 0.48377510127780526, Val loss:0.5719601869845218
9900. Training loss: 0.4837750381964308, Val loss:0.5719602274539308
9950. Training loss: 0.4837749778437354, Val loss:0.5719602661846928
10000. Training loss: 0.4837749200983357, Val loss:0.5719603032521642
```

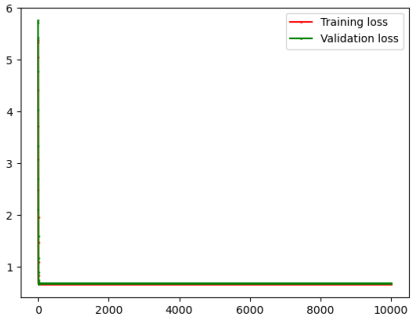


iv. L2 (reg_lambda=100)

```python
model = LinearModel(dim=X_train.shape[1], is_reg=False, loss_fn=loss_fn, act_fn=act_fn, grad_fn=grad_fn)
model.fit(X_train, y_train,lr=0.1, n_iteration =10000,val_ratio=0.2, reg_type='l2', reg_lambda=100)

# print model parameters
print("Model parameters (weights):", model.W)
# sum of absolute values of weights
print("Sum of absolute values of weights:", np.sum(np.abs(model.W)))
```

✓ 0.4s

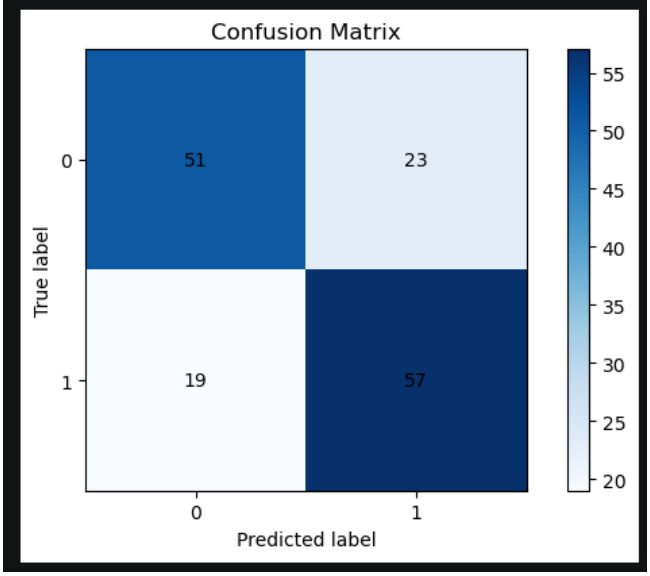
```
50. Training loss: 0.6737565170340649, Val loss:0.6828695995569816
100. Training loss: 0.6747289430162428, Val loss:0.6839071965242837
150. Training loss: 0.6747483271618208, Val loss:0.6839293732638189
200. Training loss: 0.6747487102403308, Val loss:0.6839298451300982
250. Training loss: 0.6747487179829201, Val loss:0.6839298552460086
300. Training loss: 0.6747487179798802, Val loss:0.6839298554645057
350. Training loss: 0.6747487179828773, Val loss:0.6839298554692588
400. Training loss: 0.6747487179829369, Val loss:0.6839298554693627
450. Training loss: 0.6747487179829381, Val loss:0.68392985469365
500. Training loss: 0.6747487179829382, Val loss:0.68392985469365
550. Training loss: 0.6747487179829382, Val loss:0.68392985469365
600. Training loss: 0.6747487179829382, Val loss:0.68392985469365
650. Training loss: 0.6747487179829382, Val loss:0.68392985469365
700. Training loss: 0.6747487179829382, Val loss:0.68392985469365
750. Training loss: 0.6747487179829382, Val loss:0.68392985469365
800. Training loss: 0.6747487179829382, Val loss:0.68392985469365
850. Training loss: 0.6747487179829382, Val loss:0.68392985469365
900. Training loss: 0.6747487179829382, Val loss:0.68392985469365
950. Training loss: 0.6747487179829382, Val loss:0.68392985469365
1000. Training loss: 0.6747487179829382, Val loss:0.68392985469365
1050. Training loss: 0.6747487179829382, Val loss:0.68392985469365
1100. Training loss: 0.6747487179829382, Val loss:0.68392985469365
1150. Training loss: 0.6747487179829382, Val loss:0.68392985469365
1200. Training loss: 0.6747487179829382, Val loss:0.68392985469365
1250. Training loss: 0.6747487179829382, Val loss:0.68392985469365
...
9850. Training loss: 0.6747487179829382, Val loss:0.68392985469365
9900. Training loss: 0.6747487179829382, Val loss:0.68392985469365
9950. Training loss: 0.6747487179829382, Val loss:0.68392985469365
10000. Training loss: 0.6747487179829382, Val loss:0.68392985469365
```



b. **Following 5(a), show the testing results by using the "evaluate_binary_classifier" function.**

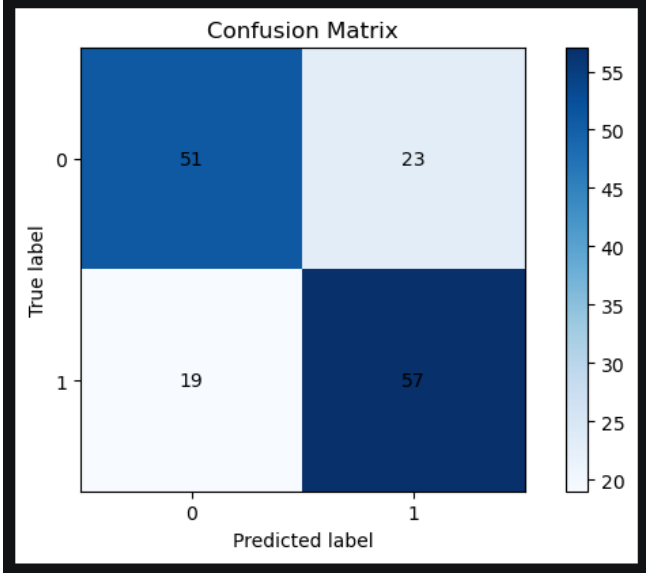
i. No regularization

```
Model Evaluation
Accuracy  : 0.7200
Precision : 0.7125
Recall    : 0.7500
F1-score  : 0.7308
```



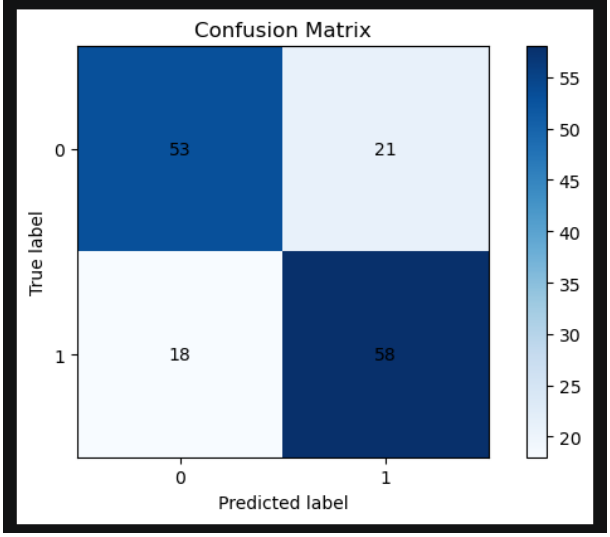
ii. L2 (reg_lambda=0.01)

```
Model Evaluation
Accuracy  : 0.7200
Precision : 0.7125
Recall    : 0.7500
F1-score  : 0.7308
```

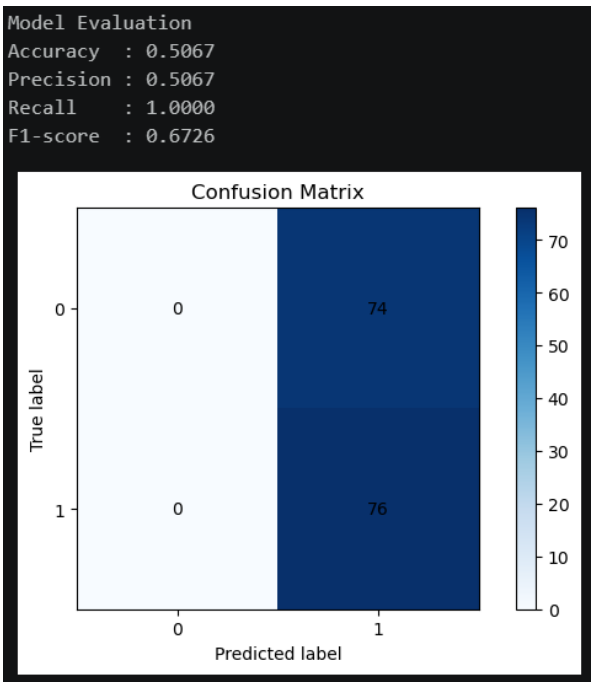


iii. L2 (reg_lambda=1)

```
Model Evaluation
Accuracy  : 0.7400
Precision : 0.7342
Recall    : 0.7632
F1-score  : 0.7484
```



iv. L2 (reg_lambda=100)



c. **Describe your observations in 5(a) and 5(b).**

1.Effect on Model Weights (Weight Decay):The most apparent impact of L2 regularization is the shrinkage of model weights. As the regularization parameter (reg_lambda) increases, the penalty on large weights becomes stronger.No Reg: Sum of absolute weights is approx. 33.84.λ = 0.01: Sum of absolute weights slightly decreases to approx. 33.25.λ = 1: Sum of absolute weights significantly drops to approx. 15.22.λ = 100: Sum of absolute weights collapses to near zero (approx. 0.43), meaning the model is heavily constrained.

2. Effect on Loss Curves:
As reg_lambda increases, the overall Training and Validation loss values also increase (e.g., from 0.46 in No Reg to 0.67 when λ = 100). This happens because the regularization penalty term dominates the loss function, restricting the model from freely minimizing the data error.

3.Effect on Model Performance (The Risk of Underfitting):Weak Regularization (λ= 0.01): The penalty is too small to make a

significant difference. The evaluation metrics (Accuracy: 0.8800) and the confusion matrix are identical to the model with no regularization.Moderate Regularization (λ= 1): The performance drops slightly (Accuracy: 0.8400). The model is constrained from perfectly fitting the training data, leading to a slight decrease in predictive accuracy.Excessive Regularization (λ= 100): The model severely underfits. Because the weights are forced to be almost zero, the model loses its ability to distinguish between classes. As seen in the confusion matrix, it simply predicts class '1' for all instances, resulting in a terrible accuracy of 0.4800.